Chapitre 3

ALGORITHMES D'OPTIMISATION

3.1 Introduction

L'objet de ce chapitre est de présenter et analyser quelques algorithmes permettant de calculer, ou plus exactement d'**approcher** les solutions de problèmes d'optimisation. Un point commun à tous ces algorithmes est qu'ils s'inspirent des conditions d'optimalité étudiées au chapitre précédent et qu'en particulier ils utilisent la connaissance des dérivées des fonctions objectifs et des contraintes. Le lecteur trouvera qu'il y a peut-être beaucoup trop d'algorithmes présentés ici, mais il doit savoir que chacun a son utilité et son efficacité en pratique sur tel ou tel problème. Aucun algorithme n'est universellement meilleur qu'un autre et le choix d'un algorithme, plutôt qu'un autre, dépend en particulier de la régularité de la fonction à minimiser (et des contraintes) et de la taille du problème à résoudre.

Ces algorithmes sont aussi tous de nature itérative : à partir d'une donnée initiale u^0, chaque méthode construit une suite $(u^n)_{n\in\mathbb{N}}$ dont nous montrerons qu'elle converge, sous certaines hypothèses, vers la solution u du problème d'optimisation considéré. Après avoir montré la **convergence de ces algorithmes** (c'est-à-dire, la convergence de la suite $(u^n)_{n\in\mathbb{N}}$ vers u quel que soit le choix de la donnée initiale u^0), nous étudierons leur vitesse de convergence.

Dans toute ce chapitre nous supposerons que la fonction objectif à minimiser J est convexe, et souvent nous ajouterons qu'elle est α-convexe, différentiable et que sa dérivée est Lipschitzienne. Ces hypothèses sont assez fortes, mais nous verrons qu'elles sont cruciales pour les démonstrations de convergence des algorithmes et qu'elles conditionnent les vitesses de convergence que l'on peut obtenir. En particulier, le rôle de la convexité est absolument essentiel : si on applique ces algorithmes à la minimisation de fonctions non convexes (ce qui arrive souvent en pratique), on peut se heurter à des difficultés sérieuses. Typiquement, ces algorithmes peuvent, au mieux converger vers un minimum local (voire vers un simple point critique), très loin d'un minimum global, au pire ne pas converger, diverger ou osciller entre plusieurs limites.

Remarque 3.1.1 Nous nous limitons aux seuls algorithmes déterministes et nous ne disons rien des algorithmes de type stochastique (recuit simulé, algorithmes génétiques, etc.). Outre le fait que leur analyse fait appel à la théorie des probabilités (que nous n'abordons pas dans ce cours), leur utilisation est très différente. Pour schématiser simplement, disons que les algorithmes déterministes sont les plus efficaces pour la minimisation de fonctions convexes, tandis que les algorithmes stochastiques permettent d'approcher des minima **globaux** (et pas seulement locaux) de fonctions non convexes (à un prix toutefois assez élevé en pratique). Nous n'évoquons pas, non plus, les algorithmes qui n'utilisent pas de dérivées car ils sont très peu efficace en pratique. •

3.2 Algorithmes de type gradient (sans contraintes)

Commençons par étudier la résolution pratique de problèmes d'optimisation en l'absence de contraintes. Soit J une fonction α-convexe différentiable définie sur l'espace de Hilbert réel V, on considère le problème sans contrainte

$$\inf_{v\in V} J(v) \ . \tag{3.1}$$

D'après le Théorème 2.3.8 il existe une unique solution u, caractérisée d'après la Remarque 2.5.2 par l'équation d'Euler $J'(u) = 0$. Pour démontrer la convergence des algorithmes nous utiliserons très souvent une hypothèse de Lipschitziannité de la dérivée J' (voir la Définition 2.4.9).

3.2.1 Algorithme de gradient à pas optimal

L'algorithme de gradient construit une suite minimisante u^n à l'aide d'une récurrence à un seul pas, c'est-à-dire que la nouvelle itérée u^{n+1} ne dépend que de la précédente u^n. On passe de u^n à u^{n+1} en suivant la ligne de plus grande pente associée à la fonction coût $J(v)$. La direction de descente correspondant à cette ligne de plus grande pente issue de u^n est donnée par l'opposé du gradient $J'(u^n)$. En effet, si l'on cherche u^{n+1} sous la forme

$$u^{n+1} = u^n - \mu^n w^n \ , \tag{3.2}$$

avec $\mu^n > 0$ petit et w^n unitaire dans V, un développement de Taylor à l'ordre 1,

$$J(u^{n+1}) = J(u^n) - \mu^n \langle J'(u^n), w^n\rangle + o(\mu^n),$$

montre que le choix de la direction $w_n = \dfrac{J'(u^n)}{\|J'(u^n)\|}$ permet de trouver la plus petite valeur de $J(u^{n+1})$ si on néglige le terme de reste (c'est-à-dire en l'absence d'autres informations comme les dérivées supérieures ou les itérées antérieures).

Ce simple constat conduit à choisir parmi les méthodes du type (3.2), qui sont appelées "méthodes de descente", la méthode de gradient ou de plus grande pente (*steepest descent* en anglais) dans laquelle w^n est positivement proportionnelle à

$J'(u^n)$. Le choix du coefficient de proportionnalité, ou pas de descente, distingue plusieurs algorithmes. Ici, nous présentons l'algorithme de **gradient à pas optimal**, dans lequel on résout une succession de problème de minimisation à une seule variable réelle (même si V n'est pas de dimension finie). A partir de u^0 quelconque dans V, on construit la suite (u^n) définie par

$$u^{n+1} = u^n - \mu^n J'(u^n) , \tag{3.3}$$

où $\mu^n \in \mathbb{R}$, appelé **pas de descente**, est choisi à chaque étape tel que

$$J(u^{n+1}) = \min_{\mu \in R} J\big(u^n - \mu J'(u^n)\big) . \tag{3.4}$$

Notons que l'on n'a pas normalisé le vecteur gradient dans (3.3). Cet algorithme converge comme l'indique le résultat suivant.

Théorème 3.2.1 *On suppose que J est α-convexe différentiable et que J' est Lipschitzien sur tout borné de V, c'est-à-dire que*

$$\forall\, M > 0,\ \exists\, L_M > 0,\ \|v\| + \|w\| \le M \ \Rightarrow\ \|J'(v) - J'(w)\| \le L_M \|v - w\| . \tag{3.5}$$

Alors l'algorithme de gradient à pas optimal converge : quel que soit u^0, la suite (u^n) définie par (3.3) et (3.4) converge vers la solution u de (3.1).

Démonstration. La fonction $j(\mu) = J\big(u^n - \mu J'(u^n)\big)$ est fortement convexe et dérivable sur $\mathbb{R}$ (si $J'(u^n) \neq 0$; sinon, on a déjà convergé, $u^n = u$!). Le problème de minimisation (3.4) a donc bien une solution unique, caractérisée par la condition $j'(\mu^n) = 0$, qui s'écrit aussi

$$\langle J'(u^{n+1}),\, J'(u^n)\rangle = 0 . \tag{3.6}$$

Ceci montre que deux "directions de descente" consécutives sont orthogonales. Puisque (3.6) implique que $\langle J'(u^{n+1}), u^{n+1} - u^n\rangle = 0$, on déduit de l'$\alpha$-convexité de J que

$$J(u^n) - J(u^{n+1}) \ge \frac{\alpha}{2}\|u^n - u^{n+1}\|^2 , \tag{3.7}$$

ce qui prouve que la suite $J(u^n)$ est décroissante. Comme elle est minorée par $J(u)$, elle converge et (3.7) montre que $u^{n+1} - u^n$ tend vers 0. D'autre part, l'α-convexité de J et le fait que la suite $J(u^n)$ est bornée montrent que la suite (u^n) est bornée : il existe une constante M telle que

$$\|u^n\| \le M .$$

Écrivant (3.5) pour $v = u^n$ et $w = u^{n+1}$ et utilisant (3.6), on obtient

$$\|J'(u^n)\|^2 \le \|J'(u^n)\|^2 + \|J'(u^{n+1})\|^2 = \|J'(u^n) - J'(u^{n+1})\|^2 \le L_M^2 \|u^{n+1} - u^n\|^2,$$

ce qui prouve que $J'(u^n)$ tend vers 0. L'α-convexité de J donne alors

$$\alpha\|u^n - u\|^2 \le \langle J'(u^n) - J'(u), u^n - u\rangle = \langle J'(u^n), u^n - u\rangle \le \|J'(u^n)\|\ \|u^n - u\| ,$$

qui implique $\alpha\|u^n - u\| \le \|J'(u^n)\|$, d'où l'on déduit la convergence de l'algorithme. □

Remarque 3.2.2 Si J est de classe C^2 sur V, alors la condition (3.5) du théorème est automatiquement vérifiée. •

Remarque 3.2.3 De manière générale on dira qu'un algorithme d'optimisation converge s'il converge quelle que soit l'initialisation u^0. En effet, cela n'aurait pas de sens de développer une théorie de convergence sous des conditions restrictives sur le vecteur initial u^0 (par exemple qu'il appartienne à un certain sous-espace vectoriel de V) car en pratique les inévitables erreurs d'arrondi du calcul numérique sur ordinateur empêcherait que ces conditions soient vérifiées exactement et la suite des itérées u^n pourrait ne pas converger. •

Remarque 3.2.4 Dans (3.2) la direction w^n, positivement proportionnelle à $J'(u^n)$, n'est pas la seule direction de descente possible, c'est-à-dire qui garantisse la décroissance de la fonction coût J pour un pas μ^n petit. Pour s'en rendre compte de manière simple, plaçons nous dans le cas où $V = \mathbb{R}^N$. Si P est une matrice symétrique définie positive de taille $N \times N$, alors $PJ'(u^n)$ est aussi une direction de descente car un développement de Taylor pour la récurrence

$$u^{n+1} = u^n - \mu^n PJ'(u^n)$$

conduit à

$$J(u^{n+1}) = J(u^n) - \mu^n \langle PJ'(u^n), J'(u^n)\rangle + o(\mu^n) < J(u^n)$$

si $\mu^n > 0$ est petit et $J'(u^n) \neq 0$. Il peut être intéressante d'utiliser un tel **préconditionnement** du gradient pour améliorer la convergence. Par exemple, si les ordres de grandeur des différentes composantes de u, et donc de $J'(u)$, sont très différents, on peut utiliser une telle matrice P diagonale pour remettre à la même échelle ces composantes. Plus généralement, le choix d'une telle matrice revient à changer le produit scalaire de V avec lequel on identifie le gradient. De ce point de vue, cette approche se généralise aux espaces de Hilbert V de dimension infinie : on change la direction de descente si on change le produit scalaire de V. •

Remarque 3.2.5 La recherche du pas optimal $\mu^n \in \mathbb{R}$ dans (3.4) est un problème très classique, appelé recherche linéaire ou en ligne. Il existe de très nombreux algorithmes pour sa résolution (voir [24]). En pratique, on n'a souvent pas besoin de trouver le pas optimal μ^n et on se contente d'un pas $\tilde{\mu}^n$ qui fait "suffisamment" décroitre la fonction $j(\mu) = J\big(u^n - \mu\, J'(u^n)\big)$. Une condition suffisante de décroissance est la règle d'Armijo-Goldstein qui, pour des paramètres $0 < p_- < p_+ < 1$ fixés, demande que le pas $\tilde{\mu}^n > 0$ vérifie

$$-p_- j'(0)\tilde{\mu}^n \leq j(0) - j(\tilde{\mu}^n) \leq -p_+ j'(0)\tilde{\mu}^n,$$

où on rappelle que $j'(0) < 0$ tant qu'on n'a pas convergé exactement. Il existe d'autres régles du même type, comme celle de Wolfe (voir [6]). •

On peut se demander si la suite des pas optimaux μ^n reste bornée, loin de 0 et de $+\infty$, ou pas. Les trois exercices suivants montrent, d'une part que c'est le cas

pour une fonction objectif quadratique ou bien α-convexe à dérivée Lipschitzienne et, d'autre part, que le pas peut tendre vers $+\infty$ pour une fonction objectif très "plate" près de son minimum.

Exercice 3.2.1 On applique l'algorithme du gradient à pas optimal à la fonction $J(x) = \frac{1}{2}Ax \cdot x - b \cdot x$, définie de $\mathbb{R}^N$ dans $\mathbb{R}$, avec $b \in \mathbb{R}^N$ et A une matrice symétrique définie positive de valeurs propres $0 < \lambda_1 \leq \cdots \leq \lambda_N$. Montrer qu'à chaque itération n le pas optimal vérifie $1/\lambda^N \leq \mu^n \leq 1/\lambda^1$.

Exercice 3.2.2 On applique l'algorithme du gradient à pas optimal à une fonction J α-convexe différentiable et telle que sa dérivée J' est L-Lipschitzienne sur V (voir la Définition 2.4.9). En utilisant l'encadrement de J par deux fonctions quadratiques (cf. la Remarque 2.4.11), montrer que la suite des pas optimaux vérifie $0 < c \leq \mu^n \leq C < +\infty$ pour deux constantes c, C indépendantes de n.

Exercice 3.2.3 Soit $J(x) = \frac{1}{2p}(Ax \cdot x)^p$, définie de $\mathbb{R}^N$ dans $\mathbb{R}$, pour un entier $p \geq 2$ et A une matrice symétrique définie positive. Montrer que J est strictement convexe, que son minimum est atteint en 0 et qu'elle n'est pas fortement convexe en 0. On applique l'algorithme du gradient à pas optimal à J et on suppose que la suite des itérées vérifie $\|x^n\| = \epsilon^n \to 0$ avec $\epsilon^n > 0$. Montrer que la suite des pas optimaux tends vers l'infini comme $\mu^n = \mathcal{O}\left((\epsilon^n)^{-2(p-1)}\right)$.

3.2.2 Algorithme de gradient à pas fixe

L'algorithme de gradient à pas fixe est une variante du précédent où l'on utilise un pas de descente fixe $\mu > 0$, indépendant du numéro d'itération n. Autrement dit, on construit une suite u^n définie par

$$u^{n+1} = u^n - \mu\, J'(u^n)\,, \tag{3.8}$$

à partir d'une initialisation $u^0 \in V$. Cette méthode est donc plus simple que l'algorithme de gradient à pas optimal, puisqu'on fait à chaque étape l'économie de la résolution de (3.4). Le résultat suivant affirme que, si le pas de descente μ est suffisamment petit, alors l'algorithme (3.8) converge.

Théorème 3.2.6 *On suppose que J est α-convexe différentiable et que J' est Lipschitzien sur V, c'est-à-dire qu'il existe une constante $L > 0$ telle que*

$$\|J'(v) - J'(w)\| \leq L\|v - w\| \quad \forall\, v, w \in V\ . \tag{3.9}$$

Alors, si $0 < \mu < 2\alpha/L^2$, l'algorithme de gradient à pas fixe converge : quel que soit u^0, la suite (u^n) définie par (3.8) converge vers la solution u de (3.1)

$$\|u^n - u\| \leq \gamma^n \|u^0 - u\| \quad \textit{avec} \quad \gamma = \sqrt{1 - 2\alpha\mu + \mu^2 L^2} < 1.$$

Démonstration. Posons $v^n = u^n - u$. Comme $J'(u) = 0$, on a $v^{n+1} = v^n - \mu\big(J'(u^n) - J'(u)\big)$, d'où il vient

$$\begin{aligned} \|v^{n+1}\|^2 &= \|v^n\|^2 - 2\mu\langle J'(u^n) - J'(u), u^n - u\rangle + \mu^2\big\|J'(u^n) - J'(u)\big\|^2 \\ &\leq \left(1 - 2\alpha\mu + L^2\mu^2\right)\|v^n\|^2 , \end{aligned} \tag{3.10}$$

d'après (3.9) et l'α-convexité. Si $0 < \mu < 2\alpha/L^2$, il est facile de voir que $1 - 2\alpha\mu + L^2\mu^2 \in]0, 1[$, et la convergence se déduit de (3.10). De manière équivalente, la même démonstration montre que l'application $v \mapsto v - \mu J'(v)$ est strictement contractante lorsque $0 < \mu < 2\alpha/L^2$, donc elle admet un unique point fixe (qui n'est autre que u) vers lequel converge la suite u^n. □

Remarque 3.2.7 L'algorithme de gradient à pas fixe est plus simple que celui à pas optimal puisqu'il ne nécessite pas de résoudre une optimisation uni-dimensionnelle (3.4) à chaque itération. Par contre, on ne connait en général pas de bonne estimation des constantes α et L et on ne sait pas a priori si un choix de pas de descente $\mu > 0$ est suffisamment petit. •

Remarque 3.2.8 Le Théorème 3.2.6 donne une estimation de la **vitesse de convergence** de l'algorithme qui est dite **géométrique** car la quantité $\|u^n - u\|^{1/n}$ admet une limite finie, inférieure à $\gamma < 1$, lorsque n tend vers $+\infty$ Cette vitesse permet de fixer le nombre d'itérations n nécessaires pour rendre l'erreur $\|u^n - u\|$ inférieure à une tolérance ϵ fixée a priori. On peut optimiser le choix du pas de descente μ dans l'intervalle $]0, 2\alpha/L^2[$ qui minimise la valeur de γ. On trouve facilement que le pas optimal est $\mu = \alpha/L^2$ qui conduit à $\gamma = \sqrt{1 - \alpha^2/L^2}$. Mais cette valeur de γ n'est pas la vitesse de convergence optimale de l'algorithme du gradient à pas fixe car on a privilégié une démonstration simple du Théorème 3.2.6. Une meilleure constante sera donnée dans la Proposition 3.3.1. Notons que, de toute façon en pratique, on ne connait souvent pas les valeurs de α et L et donc qu'il est difficile de choisir le pas optimal μ. L'Exercice 3.2.5 montre, sur un exemple quadratique, que l'algorithme du gradient à pas fixe ne peut pas converger plus vite que cette convergence géométrique (mais avec une meilleure valeur de γ). On verra par la suite d'autres algorithmes où on peut améliorer la constante γ dans la convergence géométrique (par exemple le gradient conjugué), voire où on peut améliorer la convergence géométrique qui devient quadratique (exemple de la méthode de Newton). •

Exercice 3.2.4 On se place sous les hypothèses du Théorème 3.2.6. En utilisant la définition (3.8) de l'algorithme, montrer qu'il existe une constante C_0 telle que

$$\|J'(u^n)\| \leq C_0\gamma^n,$$

et, en utilisant la convexité de J au point u^n,

$$0 \leq J(u^n) - J(u) \leq C_0\gamma^{2n} \quad \text{avec} \quad \gamma = \sqrt{1 - 2\alpha\mu + \mu^2L^2}.$$

Exercice 3.2.5 Soit A une matrice symétrique définie positive de taille $N \times N$, de valeurs propres ordonnées $0 < \lambda_1 \leq \cdots \leq \lambda_N$, et soit $b \in \mathbb{R}^N$. On considère l'algorithme du gradient à pas fixe pour la fonction définie sur $\mathbb{R}^N$ par

$$J(x) = \frac{1}{2}Ax \cdot x - b \cdot x\,.$$

Récrire l'algorithme sous la forme $(x^n - x^*) = B(x^{n-1} - x^*)$ où $x^* = A^{-1}b$ et B est une matrice que l'on précisera. En étudiant le rayon spectral de B montrer que l'algorithme converge si et seulement si le pas fixe vérifie $0 < \mu < 2/\lambda_N$, et que la meilleure vitesse de convergence de l'algorithme s'obtient pour le choix du pas de descente $\mu_B = 2/(\lambda_1+\lambda_N)$, qui conduit à

$$\|x^n - x^*\| \leq \gamma_B^n \|x^0 - x^*\| \quad \text{avec} \quad \gamma_B = \frac{1 - \lambda_1/\lambda_N}{1 + \lambda_1/\lambda_N}.$$

Comparer avec la valeur optimisée de γ dans la Remarque 3.2.8 (identifier les valeurs L et α pour la fonction $J(x)$).

La proposition suivante est une version affaiblie du Théorème 3.2.6 où on ne fait plus l'hypothèse que la fonction J est α-convexe mais seulement qu'elle est strictement convexe. De plus, on se restreint à la dimension finie car on utilise un argument de compacité.

Proposition 3.2.9 *Soit une fonction $J(u)$ définie sur $V = \mathbb{R}^N$, strictement convexe, différentiable, infinie à l'infini et telle que J' est Lipschitzien, c'est-à-dire vérifie (3.9) pour une constante $L > 0$. Alors, si $0 < \mu < 2/L$, l'algorithme de gradient à pas fixe converge : quel que soit u^0, la suite (u^n) définie par (3.8) converge vers la solution u de (3.1).*

Démonstration. Puisque $u^{n+1} = u^n - \mu J'(u^n)$ on applique le Lemme 3.2.10 avec $v = u^n$ et $v^* = u^{n+1}$, ce qui conduit à

$$J(u^{n+1}) \leq J(u^n) - \mu(1 - L\mu/2)\|J'(u^n)\|^2.$$

On en déduit que, pour $0 < \mu < 2/L$, la suite $J(u^n)$ est décroissante et, comme J est infinie à l'infini, la suite u^n est bornée. Comme $V = \mathbb{R}^N$ il existe une sous-suite convergente, notée $u^{n'}$, de la suite u^n. De la formule $u^{n'+1} - u^{n'} = -\mu J'(u^{n'})$, on déduit que $\lim_{n' \to +\infty} J'(u^{n'}) = 0$. Comme J' est continue (puisque Lipschitzien), la limite de la sous-suite $u^{n'}$ est un zéro de J'. Or, J est strictement convexe donc admet au plus un point de minimum, c'est-à-dire que J' ne s'annule qu'en un seul point u. Par conséquent, toutes les sous-suites convergentes de u^n convergent vers la même limite u. Autrement dit, toute la suite u^n converge vers u qui est l'unique point de minimum de J. □

Lemme 3.2.10 *Soit J une fonction différentiable, de dérivée L-Lipschitzienne. Soit $v \in V$ quelconque et $v^* = v - \mu J'(v)$. Alors*

$$J(v) - J(v^*) \geq \mu(1 - L\mu/2)\|J'(v)\|^2.$$

Démonstration. En vertu du Lemme 2.4.10 la fonction $J(v^*)$ est majorée par une fonction quadratique de v^*

$$J(v^*) \leq J(v) + \langle J'(v), v^* - v\rangle + \frac{L}{2}\|v^* - v\|^2.$$

On remplace $v^* - v$ par $-\mu J'(v)$ pour obtenir le résultat. □

Remarque 3.2.11 Le Lemme 3.2.10 est une garantie de décroissance d'un pas de l'algorithme du gradient, sans aucune condition de convexité sur la fonction. •

Si on ne suppose même plus que la fonction J est convexe, alors on ne peut pas garantir que la suite u^n converge vers un point de minimum.

Exercice 3.2.6 Soit une fonction $J(u)$ différentiable, infinie à l'infini et telle que J' est Lipschitzien, c'est-à-dire vérifie (3.9) pour une constante $L > 0$. On ne suppose pas que J est convexe. A l'aide des mêmes raisonnements que pour la Proposition 3.2.9, montrer que l'algorithme du gradient à pas fixe $0 < \mu < 2/L$ vérifie que $J(u^n)$ décroit et $J'(u^n)$ tends vers 0.

L'exercice suivant donne un exemple de fonction J pour lequel l'algorithme de gradient converge seulement vers un point d'inflexion et pas vers un point de minimum.

Exercice 3.2.7 Soit la fonction $J(x) = \frac{1}{2}x_1^2 + \frac{1}{4}x_2^4 - \frac{1}{2}x_2^2$ définie sur $\mathbb{R}^2$. Vérifier que J n'est pas convexe et que l'ensemble de ses points critiques (où son gradient s'annule) sont $O = (0,0)$, $P = (0,-1)$ et $Q = (0,1)$. Montrer que P et Q sont des points de minimum tandis que O est un point selle. Montrer que, si on initialise un algorithme de gradient au point $x^0 = (1,0)$, alors quelque soit la règle de choix du pas de descente $\mu^n > 0$ (fixe, optimal ou autre) qui assure la convergence la limite est le point O qui n'est pas un minimum, même local, de J.

3.3 Généralisations et autres algorithmes de type gradient

La section précédente a donné les bases de ce qu'il faut savoir sur les algorithmes de gradient pour l'optimisation sans contraintes. Cette section, qui peut être omise en première lecture, va généraliser ce que nous venons de voir dans deux directions. D'une part, nous allons voir comment les algorithmes de gradient doivent être modifiés et comment ils convergent lorsque la fonction coût à minimiser est moins régulière que ce que nous avons supposé jusqu'ici. D'autre part, nous présentons d'autres algorithmes de descente, qu'on dit du premier ordre car à chaque itération ils n'utilisent que l'information du gradient et pas celle de dérivées d'ordre supérieur (contrairement à la méthode de Newton que nous verrons plus tard). Typiquement ces nouveaux algorithmes différent des deux précédents car la nouvelle itération u^{n+1} ne dépend pas que de la solution courante u^n mais aussi de la précédente u^{n-1}

(on parle alors de méthode multi-pas). Nous en donnons quelques exemples simples pour illustrer la richesse de l'algorithmique en optimisation, sachant que certains algorithmes sont spécialisés pour des classe de fonctions coûts bien particulières.

3.3.1 Vitesse de convergence

Commençons par dire quelques mots de la vitesse de convergence de l'algorithme du gradient à pas fixe. En effet, cette question est intéressante en elle-même mais, en plus, elle motive la conception d'autres algorithmes pour l'améliorer. Comme nous l'avions dit à la Remarque 3.2.8 la vitesse de convergence du Théorème 3.2.6 n'est pas optimale. Nous donnons donc une nouvelle démonstration du Théorème 3.2.6, mais en nous plaçant en dimension finie et en supposant la fonction J de classe C^2 pour simplifier l'analyse, ce qui nous permet d'obtenir la vitesse de convergence optimale pour l'algorithme du gradient à pas fixe.

Proposition 3.3.1 *On ajoute aux hypothèses du Théorème 3.2.6 le fait que $V = \mathbb{R}^N$ et que la fonction J est de classe C^2. Alors l'algorithme de gradient à pas fixe converge pour $0 < \mu < 2/L$, c'est-à-dire que quel que soit u^0, la suite (u^n) définie par (3.8) converge vers la solution u de (3.1), et la vitesse de convergence optimale s'obtient pour le pas $\mu_{\rm opt} = 2/(L+\alpha)$*

$$\|u^n - u\| \leq \gamma_{\rm opt}^n \|u^0 - u\| \quad \textit{avec} \quad \gamma_{\rm opt} = \frac{1-\alpha/L}{1+\alpha/L} < 1.$$

Démonstration. On écrit à nouveau

$$u^{n+1} - u = u^n - u - \mu\big(J'(u^n) - J'(u)\big)$$

et on utilise un développement de Taylor avec reste exact

$$u^{n+1} - u = B^n(u^n - u) \quad \text{avec} \quad B^n = \left(\mathrm{Id} - \mu \int_0^1 J''(u + t(u^n - u))dt\right).$$

L'hypothèse (3.9) sur J' Lipschitzien implique $J'' \leq L\,\mathrm{Id}$, tandis que la forte convexité de J implique $J'' \geq \alpha\,\mathrm{Id}$. Par conséquent, la matrice symétrique B^n vérifie

$$(1-\mu L)\,\mathrm{Id} \leq B^n \leq (1-\mu\alpha)\,\mathrm{Id}.$$

Soit $\rho(B^n)$ le rayon spectral de B^n, c'est-à-dire le maximum des valeurs absolues de ses valeurs propres. On a $\rho(B^n) \leq \max(|1-\mu L|, |1-\mu\alpha|)$ dont on déduit $\rho(B^n) \leq \gamma < 1$ avec γ indépendant de n si $0 < \mu < 2/L$. Autrement dit, $\|u^n - u\| \leq \gamma^n \|u^0 - u\|$ et l'algorithme du gradient à pas fixe converge si $0 < \mu < 2/L$. Si maintenant on choisit μ tel que $(1-\mu L) \leq 0 \leq (1-\mu\alpha)$, alors

$$\rho(B^n) \leq \max(\mu L - 1, 1 - \mu\alpha)$$

et dans cette majoration la valeur optimale du pas est $\mu_{\rm opt} = 2/(L+\alpha)$ qui conduit à $\gamma_{\rm opt} = \frac{L-\alpha}{L+\alpha}$. □

Remarque 3.3.2 Comme l'a montré l'Exercice 3.2.5 sur une fonction quadratique, on ne peut pas espérer améliorer la vitesse de convergence de l'algorithme du gradient à pas fixe obtenue à la Proposition 3.3.1, qui est donc bien optimale. De même, l'Exercice 3.2.5 montre que l'on ne peut pas élargir l'ensemble des pas, $0 < \mu < 2/L$, pour lesquels l'algorithme du gradient à pas fixe converge. •

Si la fonction J que l'on minimise n'est pas fortement (ou α-) convexe, alors la vitesse de convergence est bien moindre, devenant algébrique au lieu de géométrique.

Proposition 3.3.3 *Soit une fonction $J(u)$ définie sur $V = \mathbb{R}^N$, strictement convexe, de classe C^2, infinie à l'infini et telle que J' est Lipschitzien, c'est-à-dire vérifie (3.9) pour une constante $L > 0$. Alors l'algorithme de gradient à pas fixe converge pour $0 < \mu < 2/L$, c'est-à-dire que quel que soit u^0, la suite (u^n) définie par (3.8) converge vers la solution u de (3.1), et on a*

$$0 \leq J(u^n) - J(u) \leq \frac{\kappa}{n+1} \qquad \text{avec } \kappa = \frac{\|u^0 - u\|^2}{\mu(1 - L\mu/2)}.$$

Remarque 3.3.4 Noter que la Proposition 3.3.3 ne dit rien de la vitesse de convergence de $\|u^n - u\|$ et que la vitesse de convergence est ici définie comme la vitesse de convergence de $J(u^n)$ vers $J(u)$. Cette dernière est **algébrique** (comme $1/n$), ce qui est beaucoup moins rapide que la convergence géométrique du cas fortement convexe (comme γ^{2n} avec $0 < \gamma < 1$, d'après l'Exercice 3.2.4). On ne peut pas avoir une estimation de $\|u^n - u\|$, indépendante de J, car les fonctions convexes, mais pas fortement convexes, peuvent être très plates près de leur minimum et la convergence de u^n vers u peut être aussi lente que l'on veut selon la forme de J. •

Démonstration. On a déjà démontré à la Proposition 3.2.9 que u^n converge vers l'unique point de minimum u de J. On calcule la norme au carré de la relation

$$u^{n+1} - u = u^n - u - \mu \left(J'(u^n) - J'(u) \right),$$

où l'on a utilisé la propriété $J'(u) = 0$. On obtient

$$\|u^{n+1} - u\|^2 = \|u^n - u\|^2 + \mu^2 \|J'(u^n) - J'(u)\|^2 - 2\mu \langle J'(u^n) - J'(u), u^n - u \rangle$$

et l'on majore le dernier terme à l'aide du Lemme 3.3.5 ci-dessous. Comme $J'(u) = 0$, on en déduit

$$\|u^{n+1} - u\|^2 \leq \|u^n - u\|^2 - 2\mu(1 - L\mu/2) \|J'(u^n)\|^2,$$

et donc que la suite $\|u^n - u\|$ est décroissante pour $0 < \mu < 2/L$. Par ailleurs, la convexité de J en u^n donne

$$J(u^n) + \langle J'(u^n), u - u^n \rangle \leq J(u),$$

et par Cauchy-Schwarz ainsi que par la décroissance de $\|u^n - u\|$

$$0 \leq J(u^n) - J(u) \leq \|J'(u^n)\| \, \|u^n - u\| \leq \|J'(u^n)\| \, \|u^0 - u\|. \tag{3.11}$$

Or, lors de la preuve la Proposition 3.2.9 on a montré que

$$J(u^{n+1}) \leq J(u^n) - \mu(1 - L\mu/2)\|J'(u^n)\|^2,$$

c'est-à-dire, en posant $\Delta^n = J(u^n) - J(u) > 0$, que

$$\mu(1 - L\mu/2)\|J'(u^n)\|^2 \leq \Delta^n - \Delta^{n+1} \tag{3.12}$$

En combinant (3.11) et (3.12) on obtient

$$\Delta^{n+1} \leq \Delta^n - \kappa^{-1}(\Delta^n)^2 \quad \text{avec } \kappa = \frac{\|u^0 - u\|^2}{\mu(1 - L\mu/2)}. \tag{3.13}$$

Montrons alors par récurrence sur n que $\Delta^n \leq \frac{\kappa}{n+1}$, ce qui est le résultat recherché. Pour $n = 0$, on reprend (3.11) et on majore

$$\|J'(u^0)\| = \|J'(u^0) - J'(u)\| \leq L\|u^0 - u\| \leq \frac{\|u^0 - u\|}{\mu(1 - L\mu/2)}$$

car $0 < \mu < 2/L$, donc $\Delta^0 \leq \kappa$. De (3.13) on a immédiatement $\Delta^{n+1} \leq \Delta^n \leq \Delta^0 \leq \kappa$. Supposons qu'à l'ordre $(n-1)$ on ait bien $n\Delta^{n-1} \leq \kappa$. On calcule alors

$$(n+1)\Delta^n \leq (n+1)\Delta^{n-1} - (n+1)\kappa^{-1}(\Delta^{n-1})^2 \leq n\Delta^{n-1} + \Delta^{n-1}\left(1 - (n+1)\kappa^{-1}\Delta^{n-1}\right)$$

$$\leq \kappa + \kappa^{-1}\Delta^{n-1}\left(\kappa - (n+1)\Delta^n\right)$$

puisque $\Delta^n \leq \Delta^{n-1}$. On regroupe les termes pour en déduire

$$0 \leq \left(\kappa - (n+1)\Delta^n\right)\left(1 + \kappa^{-1}\Delta^{n-1}\right)$$

qui prouve que $\kappa - (n+1)\Delta^n \geq 0$ comme voulu. □

Lemme 3.3.5 *Soit une fonction $J(u)$ définie sur $V = \mathbb{R}^N$, strictement convexe, de classe C^2 et telle que J' est Lipschitzien, c'est-à-dire vérifie (3.9) pour une constante $L > 0$. Alors*

$$\|J'(v) - J'(w)\|^2 \leq L\langle J'(v) - J'(w), v - w\rangle \quad \textit{pour tout } v, w \in V.$$

Démonstration. On écrit la formule de Taylor avec reste exact

$$J'(v) = J'(w) + \int_0^1 J''(w + t(v - w))(v - w)\, dt.$$

Comme J est strictement convexe et J' L-Lipschitzien, la matrice $N \times N$ des dérivées secondes est inversible et vérifie $0 < J''(u) \leq L\,\mathrm{Id}$ pour tout $u \in \mathbb{R}^N$. Par conséquent, il en est de même de la matrice

$$A = \int_0^1 J''(w + t(v - w))\, dt$$

qui vérifie $A^{-1} \geq 1/L\,\mathrm{Id}$. On déduit alors de la formule de Taylor ci-dessus

$$A^{-1}\left(J'(v) - J'(w)\right) = v - w,$$

donc, pour tout $v, w \in \mathbb{R}^N$,

$$\frac{1}{L}\|J'(v) - J'(w)\|^2 \leq \langle A^{-1}\left(J'(v) - J'(w)\right), J'(v) - J'(w)\rangle = \langle J'(v) - J'(w), v - w\rangle.$$

En fait, le résultat reste vrai pour une fonction $J(u)$ convexe mais pas strictement convexe : il suffit de lui ajouter $\epsilon\|u\|^2$ avec $\epsilon > 0$ puis de passer à la limite $\epsilon \to 0$ dans le résultat. □

Remarque 3.3.6 Un aspect essentiel des estimations de vitesse de convergence que nous venons de voir est qu'elles sont toutes indépendantes de la dimension de l'espace V dans lequel on minimise. C'est particulièrement évident lorsque les preuves sont faites pour un espace de Hilbert quelconque, possiblement de dimension infinie. C'est une information très importante lorsqu'on travaille sur des problèmes en très grande dimension (voire infinie). Par contre, en général le coût de calcul d'une itération dépend de la dimension. Penser, par exemple, à la minimisation d'une fonction quadratique comme dans l'Exercice 3.2.5, où le coût d'une itération de la méthode du gradient à pas fixe est dominée par le calcul du gradient qui correspond à un produit matrice vecteur et est donc de l'ordre de N^2, avec N la dimension. Néanmoins, il faut se méfier de certains problèmes où les constantes α et L qui apparaissent dans la vitesse de convergence dépendent, de manière plus ou moins cachée, de la dimension (par exemple, pour une fonction quadratique où la matrice est issue de la discrétisation d'une équation différentielle). •

3.3.2 Algorithme de Nesterov

Nesterov [23] a proposé un algorithme qui améliore la vitesse de convergence de la méthode du gradient à pas fixe pour les fonctions convexes. Pour cette raison l'algorithme de Nesterov est parfois appelé **algorithme du gradient accéléré**. On l'initialise avec une constante $a_0 = 1$ et un vecteur $v^0 = u^{-1} \in V$. Ensuite, pour $n \geq 0$,

$$\left\{\begin{array}{l} u^n = v^n - \mu\, J'(v^n) \\ a_{n+1} = \dfrac{1}{2}(1 + \sqrt{4a_n^2 + 1}) \\ v^{n+1} = u^n + \dfrac{a_n - 1}{a_{n+1}}(u^n - u^{n-1}) \end{array}\right. \tag{3.14}$$

avec un pas fixe $\mu > 0$. L'idée de cet algorithme est d'extrapoler un nouveau vecteur v^{n+1} au delà du segment (u^{n-1}, u^n) où l'on calcule le gradient qui donnera la nouvelle itérée u^{n+1} (on parle aussi de sur-relaxation). La formule de récurrence pour le coefficient a_n vient d'une majoration précise dans la démonstration de convergence. On vérifie facilement que, pour n grand, $a_n \approx 1 + n/2$ et le coefficient d'extrapolation vaut $(a_n - 1)/a_{n+1} \approx 1$.

Proposition 3.3.7 *Soit J une fonction convexe différentiable, telle que J' est L-Lipschitzienne, c'est-à-dire vérifie (3.9). Soit un pas de descente μ tel que $0 < \mu \leq 1/L$. On suppose qu'il existe un point de minimum u de J sur V. Alors la suite u^n de l'agorithme de Nesterov, définie par* (3.14)*, vérifie*

$$0 \leq J(u^n) - J(u) \leq \frac{\kappa}{(n+2)^2} \qquad \text{avec } \kappa = 2\mu^{-1}\|u^0 - u\|^2 + 4(J(u^0) - J(u)).$$

Remarque 3.3.8 Avec quelques hypothèses supplémentaires on peut montrer que la suite u^n converge vers u (voir l'Exercice 3.3.1 ci-dessous). La vitesse de convergence de la Proposition 3.3.7 améliore sensiblement ($1/n^2$ au lieu de $1/n$) celle obtenue dans la Proposition 3.3.3. Comme on le verra à la Remarque 3.3.9 cette vitesse est optimale pour les fonctions convexes. Evidemment pour des fonctions fortement convexes, quitte à changer la définition précise du coefficient d'extrapolation a_n, on peut encore améliorer cette vitesse de convergence et retrouver une convergence géométrique (voir la Proposition 3.3.10). •

Démonstration. Soit $p^n = (a_n - 1)(u^{n-1} - u^n)$ de sorte que $v^{n+1} = u^n - p^n/a_{n+1}$. L'astuce de Nesterov est d'étudier la convergence de la suite $u^n - p^n$ plutôt que celle de u^n ou v^n. On calcule

$$u^{n+1} - p^{n+1} = u^{n+1} - (a_{n+1} - 1)(u^n - u^{n+1}) = a_{n+1}u^{n+1} - (a_{n+1} - 1)u^n$$

$$= a_{n+1}(v^{n+1} - \mu J'(v^{n+1})) - (a_{n+1} - 1)u^n = u^n - p^n - a_{n+1}\mu J'(v^{n+1}).$$

On en déduit

$$\begin{aligned} \|u^{n+1} - p^{n+1} - u\|^2 &= \|u^n - p^n - u\|^2 + a_{n+1}^2\mu^2\|J'(v^{n+1})\|^2 \\ &\quad +2a_{n+1}\mu\langle J'(v^{n+1}), u - v^{n+1}\rangle \\ &\quad +2(a_{n+1} - 1)\mu\langle J'(v^{n+1}), p^n\rangle, \end{aligned} \tag{3.15}$$

en ayant utilisé $a_{n+1}(v^{n+1} + p^n - u^n) = (a_{n+1} - 1)p^n$ dans la dernière ligne. On majore la deuxième ligne de (3.15) par convexité de J

$$\langle J'(v^{n+1}), u - v^{n+1}\rangle \leq J(u) - J(v^{n+1})$$

et la troisième ligne de même car

$$\langle J'(v^{n+1}), p^n\rangle = a_{n+1}\langle J'(v^{n+1}), u^n - v^{n+1}\rangle \leq a_{n+1}(J(u^n) - J(v^{n+1})).$$

Ainsi (3.15) devient

$$\begin{aligned} \|u^{n+1} - p^{n+1} - u\|^2 &\leq \|u^n - p^n - u\|^2 + a_{n+1}^2\mu^2\|J'(v^{n+1})\|^2 \\ &\quad +2a_{n+1}\mu(J(u) - J(v^{n+1})) \\ &\quad +2(a_{n+1} - 1)\mu a_{n+1}(J(u^n) - J(v^{n+1})). \end{aligned} \tag{3.16}$$

On majore les deux dernières lignes de (3.16) en écrivant

$$\begin{aligned} J(u) - J(v^{n+1}) &= J(u) - J(u^{n+1}) + J(u^{n+1}) - J(v^{n+1}) \\ J(u^n) - J(v^{n+1}) &= J(u^n) - J(u^{n+1}) + J(u^{n+1}) - J(v^{n+1}) \end{aligned}$$

puis en applicant le Lemme 3.2.10 avec $v = v^{n+1}$ et $v^* = u^{n+1}$

$$J(u^{n+1}) - J(v^{n+1}) \leq -\mu(1 - L\mu/2)\|J'(v^{n+1})\|^2. \tag{3.17}$$

Par conséquent, (3.16) devient

$$\begin{aligned} \|u^{n+1} - p^{n+1} - u\|^2 &\leq \|u^n - p^n - u\|^2 + a_{n+1}^2\mu^2\|J'(v^{n+1})\|^2(L\mu - 1) \\ &\quad +2\mu a_{n+1}(J(u) - J(u^{n+1})) \\ &\quad +2\mu(a_{n+1} - 1)a_{n+1}(J(u^n) - J(u^{n+1})). \end{aligned} \tag{3.18}$$

On remarque que la définition de la suite a_n implique que $(a_{n+1} - 1)a_{n+1} = a_n^2$. En supposant que $0 < \mu \leq 1/L$ et en introduisant $J(u)$ dans la dernière ligne de (3.18), on obtient

$$\|u^{n+1} - p^{n+1} - u\|^2 + 2\mu a_{n+1}^2(J(u^{n+1}) - J(u)) \leq \|u^n - p^n - u\|^2 + 2\mu a_n^2(J(u^n) - J(u))$$

donc, comme $a_0 = 1$ et $p^0 = 0$,

$$2\mu a_n^2(J(u^n) - J(u)) \leq \|u^0 - u\|^2 + 2\mu(J(u^0) - J(u)).$$

Il est facile de vérifier que $a_n \geq 1 + n/2$ ce qui donne la majoration attendue. □

Exercice 3.3.1 On se place sous les hypothèses de la Proposition 3.3.7 et dans un espace de dimension finie $V = \mathbb{R}^N$. On suppose de plus que la fonction J est strictement convexe et infinie à l'infini.On considère la suite u^n donnée par l'algorithme (3.14) de Nesterov.

1. Montrer que la suite u^n est bornée.
2. Montrer que toute sous-suite convergente de u^n converge vers une limite où J' s'annule.
3. En déduire que toute la suite u^n converge vers l'unique point de minimum de J.

Remarque 3.3.9 La vitesse de convergence de la Proposition 3.3.7 est optimale pour des fonctions convexes, mais pas fortement convexes, et à dérivée Lipschtzienne. Précisément, pour tout entier n (plus petit que la dimension de l'espace V), on peut construire une fonction J et un point initial u^0 tel qu'à l'itération n la différence $J(u^n) - J(u)$ soit précisément de l'ordre de la borne supérieure donnée par la Proposition 3.3.7. En effet, pour un entier $p \leq N$, considérons l'exemple suivant d'une fonction J définie sur $\mathbb{R}^N$

$$J(v) = \frac{L}{8}\left((v_1 - 1)^2 + \sum_{i=2}^{p}(v_i - v_{i-1})^2 + \sum_{i=p+1}^{N} v_i^2\right)$$

dont l'unique point de minimum est $u = (1, ..., 1, 0, ..., 0)$, qui a ses p premières composantes égales à 1 et les suivantes égales à 0, avec $J(u) = 0$. On initialise un algorithme du gradient avec $u^0 = 0$. Tous les algorithmes de gradient que nous avons étudiés (pas optimal ou fixe, boule pesante, Nesterov ou gradient conjugué) vérifient la propriété suivante : à chaque itération n, la solution approchée u^n appartient à l'espace affine $u^0 + E_n$ où E_n est le sous-espace vectoriel engendré par $\{J'(u^0), J'(u^1), ..., J'(u^{n-1})\}$. Par conséquent, une borne inférieure est

$$J(u^n) \geq \min_{v \in u^0 + E_n} J(v).$$

Or, dans l'exemple ci-dessus, il est facile de voir, par récurrence sur n, que $J'(u^{n-1})$ appartient à l'espace engendré par les n premiers vecteurs de la base canonique, $\{e_1, ..., e_n\}$, et par conséquent que $E_n \subset \text{Vect}\{e_1, ..., e_n\}$ donc

$$J(u^n) \geq \min_{v \in u^0 + E_n} J(v) \geq \min_{v \in u^0 + \text{Vect}\{e_1, ..., e_n\}} J(v).$$

Autrement dit, sur cet exemple (très défavorable !) l'algorithme de gradient (quelle que soit sa version) ne peut propager l'information que la solution u a ses p premières composantes égales à 1, à partir d'une initialisation nulle $u^0 = 0$, qu'à la "vitesse" d'une composante par itération. Un calcul facile permet de calculer le minimum sur le sous-espace affine $u^0 + \text{Vect}\{e_1, ..., e_n\}$: ce point de minimum, noté w^n, a ses dernières $N - n$ composantes nulles tandis que les n premières sont $w_i^n = 1 - \frac{i}{n+1}$ pour $1 \leq i \leq n$. Par conséquent $(w_1^n - 1)^2 = (w_i^n - w_{i-1}^n)^2 = (w_n^n)^2 = \frac{1}{(n+1)^2}$ pour $2 \leq i \leq n$, ce qui donne, pour $0 \leq n \leq p$,

$$\min_{v \in u^0 + \text{Vect}\{e_1, ..., e_n\}} J(v) = J(w^n) = \frac{L}{8(n+1)}.$$

Comme $J(u) = 0$ et $\|u^0 - u\|^2 = p$, on a donc

$$J(u^n) - J(u) \geq \frac{L}{8(n+1)} = \frac{L}{8p(n+1)} \|u^0 - u\|^2,$$

ce qui, à l'itération $n = p - 1$, donne

$$J(u^n) - J(u) \geq \frac{L}{8(n+1)^2} \|u^0 - u\|^2.$$

Or la borne supérieure de la Proposition 3.3.7 est

$$J(u^n) - J(u) \leq \frac{\kappa}{(n+2)^2} \quad \text{avec } \kappa = 2\mu^{-1}\|u^0 - u\|^2 + 4(J(u^0) - J(u)),$$

on en déduit qu'à une constante multiplicative près (indépendante de n) on ne peut pas améliorer cette borne supérieure. •

L'algorithme de Nesterov [23] peut être amélioré pour des fonctions fortement convexes en changeant les formules d'extrapolation, du moins si l'on connait les

valeurs des constantes α (convexité) et L (Lipschitziannité), ce qui est malheureusement rarement le cas en pratique. En posant $q = \alpha/L < 1$, l'algorithme de Nesterov devient, pour $n \geq 0$,

$$\left\{ \begin{array}{l} a_{n+1} = \dfrac{1 - qa_n^2 + \sqrt{(1-qa_n^2)^2 + 4a_n^2}}{2} \\ v^n = u^n + \dfrac{(a_n - 1)(1 - a_{n+1}\alpha\mu)}{a_{n+1}(1-\alpha\mu)}(u^n - u^{n-1}) \\ u^{n+1} = v^n - \mu\, J'(v^n), \end{array} \right. \tag{3.19}$$

que l'on initialise avec $u^0 = u^{-1} \in V$ et $1 \leq a_0 \leq 1/\sqrt{q}$.

Proposition 3.3.10 *On suppose que J est α-convexe différentiable, de dérivée L-Lipschitzienne et que le pas de descente vérifie $0 < \mu \leq 1/L$. Alors la suite u^n de l'agorithme de Nesterov, définie par* (3.19)*, converge vers la solution u de (3.1), et, pour $\mu = 1/L$, la vitesse de convergence est*

$$\|u^n - u\| \leq \frac{2}{\alpha}\gamma_{\text{Nest}}^n \|u^0 - u\| \quad \textit{avec} \quad \gamma_{\text{Nest}} = \left(1 - \sqrt{\frac{\alpha}{L}}\right).$$

Remarque 3.3.11 La vitesse de convergence de l'algorithme de Nesterov est asymptotiquement (c'est-à-dire quand le rapport α/L est petit) meilleure que celle du gradient à pas optimal. Nous verrons d'autres algorithmes (boule pesante, gradient conjugué) qui ont le même ordre de grandeur asymptotique en $q = \alpha/L$ de leur vitesse de convergence. Lorsque n tends vers l'infini, la suite a_n tends vers $1/\sqrt{q}$ et, pour $\mu = 1/L$, le coefficient d'extrapolation qui donne v^n dans (3.19) converge vers $(1 - \sqrt{q})/(1 + \sqrt{q})$. •

Démonstration. On applique l'inégalité (3.21) du Lemme 3.3.12 à

$$w = \frac{t-1}{t}u^n + \frac{1}{t}u, \quad v = v^n, \quad v^* = u^{n+1} = v^n - \mu\, J'(v^n),$$

avec $t \geq 1$. En utilisant aussi la forte convexité de J

$$J(w) \leq \frac{t-1}{t}J(u^n) + \frac{1}{t}J(u) - \frac{\alpha}{2}\frac{(t-1)}{t^2}\|u^n - u\|^2,$$

en multipliant (3.21) par t^2 il vient

$$t(t-1)J(u^n) + tJ(u) - \frac{\alpha}{2}(t-1)\|u^n - u\|^2 + \frac{1-\alpha\mu}{2\mu}\|(t-1)u^n + u - tv^n\|^2$$

$$\geq t^2 J(u^{n+1}) + \frac{1}{2\mu}\|tu^{n+1} - (t-1)u^n - u\|^2$$

que l'on réécrit

$$t(t-1)(J(u^n) - J(u)) - \frac{\alpha}{2}(t-1)\|u^n - u\|^2 + \frac{1-\alpha\mu}{2\mu}\|u - u^n + t(u^n - v^n)\|^2$$

$$\geq t^2(J(u^{n+1}) - J(u)) + \frac{1}{2\mu}\|u^{n+1} - u - (t-1)(u^n - u^{n+1})\|^2.$$

Or

$$-\frac{\alpha}{2}(t-1)\|u^n - u\|^2 + \frac{1-\alpha\mu}{2\mu}\|u - u^n + t(u^n - v^n)\|$$
$$= \frac{1-t\alpha\mu}{2\mu}\|u^n - u\|^2 + \frac{1-\alpha\mu}{2\mu}t^2\|u^n - v^n\|^2 + \frac{1-\alpha\mu}{\mu}t\langle u - u^n, u^n - v^n\rangle$$
$$= \frac{1-t\alpha\mu}{2\mu}\|u - u^n + \frac{1-\alpha\mu}{1-t\alpha\mu}t(u^n - v^n)\|^2 + \frac{t^2(1-t)\mu(1-\alpha\mu)}{2(1-t\alpha\mu)}\|u^n - v^n\|^2.$$

Par conséquent

$$\begin{aligned} &t(t-1)(J(u^n) - J(u)) + \frac{1-t\alpha\mu}{2\mu}\|u - u^n + \frac{1-\alpha\mu}{1-t\alpha\mu}t(u^n - v^n)\|^2 \\ &\geq t^2(J(u^{n+1}) - J(u)) + \frac{1}{2\mu}\|u^{n+1} - u - (t-1)(u^n - u^{n+1})\|^2 \\ &+\frac{t^2(t-1)\mu(1-\alpha\mu)}{2(1-t\alpha\mu)}\|u^n - v^n\|^2. \end{aligned} \tag{3.20}$$

On choisit alors $t = a_{n+1}$, où a_n est la suite définie par (3.19). Grâce aux propriétés du Lemme 3.3.13, on a $a_{n+1} \geq 1$ ainsi que

$$a_{n+1}(a_{n+1} - 1) \leq a_n^2(1 - qa_{n+1}) \leq a_n^2(1 - a_{n+1}\alpha\mu),$$

puisque $\alpha\mu \leq q$, et le dernier terme de (3.20) est positif. Prenant en compte la définition de v^n

$$v^n = u^n + \frac{(a_n - 1)(1 - a_{n+1}\alpha\mu)}{a_{n+1}(1 - \alpha\mu)}(u^n - u^{n-1})$$

on déduit alors de (3.20)

$$(1 - a_{n+1}\alpha\mu)\Delta_n \geq \Delta_{n+1},$$

avec

$$\Delta_n = a_n^2\,(J(u^n) - J(u)) + \frac{1}{2\mu}\|u - u^n - (a_n - 1)(u^n - u^{n-1})\|^2,$$

c'est-à-dire que

$$\Delta_n \leq \Delta_0 \prod_{k=1}^{n}(1 - a_k\alpha\mu).$$

Comme $0 \leq 1 - a_k\alpha\mu \leq 1$ et que $\lim_{n\to+\infty} a_n = 1/\sqrt{q}$, on en déduit que Δ_n tends vers 0 et l'algorithme converge. De plus, si on choisit le pas $\mu = 1/L$, comme $1 - \frac{1}{a_n} = (1 - qa_n)\frac{a_{n-1}^2}{a_n^2}$, on a

$$\prod_{k=1}^{n}(1 - a_k\alpha\mu) = \prod_{k=1}^{n}(1 - qa_k) = \frac{a_n^2}{a_0^2}\prod_{k=1}^{n}(1 - \frac{1}{a_k}) \leq a_n^2(1 - \sqrt{q})^n,$$

d'où l'on déduit en particulier $J(u^n) - J(u) \leq\leq (1-\sqrt{q})^n$. Alors la forte convexité de J et le fait que $J'(u) = 0$ donne

$$\frac{\alpha}{2}\|u^n - u\|^2 \leq J(u^n) - J(u) \leq (1-\sqrt{q})^n,$$

ce qui conclut la démonstration. □

Lemme 3.3.12 *Soit J une fonction α-convexe sur V dont la dérivée J' est Lipschitzienne de constante L. Pour tout $v, w \in V$ et pour tout $0 < \mu \leq 1/L$, en notant $v^* = v - \mu J'(v)$, on a*

$$J(w) + \frac{1-\alpha\mu}{2\mu}\|w - v\|^2 \geq J(v^*) + \frac{1}{2\mu}\|v^* - w\|^2. \tag{3.21}$$

Démonstration. Pour tout $v, w \in V$ la forte convexité de J s'écrit

$$J(w) \geq J(v) + \langle J'(v), w - v\rangle + \frac{\alpha}{2}\|w - v\|^2 \tag{3.22}$$

auquel on ajoute un terme $\|w - v\|^2/(2\mu)$ de part et d'autre pour obtenir

$$J(w) + \frac{1-\alpha\mu}{2\mu}\|w - v\|^2 \geq J(v) + \langle J'(v), w - v\rangle + \frac{1}{2\mu}\|w - v\|^2. \tag{3.23}$$

On note $Q(w)$ la fonction convexe quadratique à droite de (3.23)

$$Q(w) = J(v) + \langle J'(v), w - v\rangle + \frac{1}{2\mu}\|w - v\|^2$$

qui admet $v^* = v - \mu J'(v)$ comme unique point de minimum. Puisque $Q(w)$ est quadratique et que $Q'(v^*) = 0$, on a

$$Q(w) = Q(v^*) + \frac{1}{2\mu}\|v^* - w\|^2. \tag{3.24}$$

En combinant (3.23) et (3.24) on obtient

$$J(w) + \frac{1-\alpha\mu}{2\mu}\|w - v\|^2 \geq J(v) + \langle J'(v), v^* - v\rangle + \frac{1}{2\mu}\|v^* - v\|^2 + \frac{1}{2\mu}\|v^* - w\|^2. \tag{3.25}$$

Mais comme J' est L-Lipschitzien, en vertu du Lemme 2.4.10 on a

$$J(v^*) \leq J(v) + \langle J'(v), v^* - v\rangle + \frac{L}{2}\|v^* - v\|^2. \tag{3.26}$$

Donc (3.25) et (3.26) conduisent au résultat

$$\begin{aligned} J(w) + \frac{1-\alpha\mu}{2\mu}\|w - v\|^2 &\geq J(v^*) - \frac{L}{2}\|v^* - v\|^2 + \frac{1}{2\mu}\|v^* - v\|^2 + \frac{1}{2\mu}\|v^* - w\|^2 \\ &\geq J(v^*) + \frac{1}{2\mu}\|v^* - w\|^2 \end{aligned}$$

car $\mu \leq 1/L$. □

Lemme 3.3.13 *Soit la suite définie par*

$$a_{n+1} = \frac{1 - qa_n^2 + \sqrt{(1-qa_n^2)^2 + 4a_n^2}}{2},$$

avec $a_0\sqrt{q} \leq 1$. Alors a_n est une suite croissante et majorée, $a_n\sqrt{q} \leq 1$, qui converge vers $1/\sqrt{q}$. En particulier, si $a_0 \geq 1$, alors $a_n \geq 1$ pour tout n. Par ailleurs, elle vérifie $a_{n+1}(a_{n+1}-1) \leq a_n^2(1-qa_{n+1})$.

Remarque 3.3.14 Si $q = 0$, on retrouve la suite a_n définie par (3.14) qui a un comportement différent puisque $a_n \approx 1 + n/2$. Par ailleurs, si $a_0 = 1/\sqrt{q}$, alors la suite est constante $a_n = 1/\sqrt{q}$. •

Démonstration. Montrons par récurrence que $a_n\sqrt{q} \leq 1$. La propriété est vraie pour $n = 0$. Pour $n \geq 0$, le coefficient a_{n+1} est la racine positive du polynôme

$$a_{n+1}^2 - (1 - qa_n^2)a_{n+1} - a_n^2 = 0.$$

En multipliant par q on en déduit

$$qa_{n+1}^2 = qa_{n+1} + (1 - qa_{n+1})qa_n^2. \tag{3.27}$$

Si $qa_{n+1} \geq 1$, on en déduirait $qa_{n+1}^2 \leq qa_{n+1}$, c'est-à-dire $qa_{n+1} \leq q < 1$, ce qui est contradictoire avec l'hypothèse. Donc $qa_{n+1} < 1$ et (3.27) implique que qa_{n+1}^2 est une combinaison convexe de 1 et qa_n^2. Alors, par concavité de la racine carrée, on obtient que $\sqrt{q}a_{n+1} \leq 1$ si $\sqrt{q}a_n \leq 1$. Par ailleurs, on peut aussi réécrire (3.27) sous la forme

$$a_{n+1}^2 = a_n^2 + (1 - qa_n^2)a_{n+1} \geq a_n^2,$$

donc la suite a_n est croissante. Comme elle est majorée, elle converge et sa limite se déduit facilement de (3.27). Finalement,

$$a_{n+1}(a_{n+1}-1) = a_n^2(\frac{1}{a_n} - qa_{n+1}) \leq a_n^2(1 - qa_{n+1})$$

car $a_n \geq a_0 \geq 1$. □

3.3.3 Algorithme de la boule pesante

Nous présentons ici un algorithme qui est intéressant en lui-même mais qui est surtout l'occasion de montrer une source d'inspiration dans la conception des algorithmes d'optimisation en les interprétant comme des schémas de discrétisation en temps de systèmes dynamiques. Par exemple, l'algorithme de gradient à pas fixe (3.8) peut se voir comme un schéma explicite pour l'équation différentielle

$$\dot{u}(t) = -J'(u(t)), \tag{3.28}$$

avec la donnée initiale $u(0) = u^0$ (la notation $\dot{u}$ signifie la dérivée par rapport à la variable de temps t). En effet, si on assimile le pas de descente $\mu > 0$ à un pas de

temps, alors u^n peut être interprété comme une approximation au temps $t^n = n\mu$ de la solution $u(t)$ (voir [1] pour des détails sur cette notion). L'équation différentielle ordinaire (3.28) modélise la trajectoire d'un point qui descendrait le long du potentiel $J(u)$ (on peut vérifier que $J(u(t))$ est décroissant le long de la trajectoire ; c'est ce qu'on appelle une fonction de Lyapunov). Si la fonction J est fortement convexe, on peut aussi montrer que la trajectoire $u(t)$ converge, quand t tends vers l'infini, vers l'unique point de minimum de J.

Expliquons maintenant la motivation et l'origine de l'algorithme de la boule pesante dans ce contexte. Si la fonction J n'est pas convexe et présente des minima locaux, la solution $u(t)$ peut converger vers un tel minimum local, loin d'un minimum global. Pour améliorer cette situation défavorable, une idée est de changer l'équation différentielle, pour y rajouter un terme d'inertie, qui permettrait au point matériel solution de cette nouvelle équation de sortir d'un minimum local en remontant la pente grâce à l'inertie acquise. Autrement dit, l'équation différentielle de cette boule pesante serait

$$m\ddot{u}(t) + \dot{u}(t) = -J'(u(t)),$$

où $m > 0$ serait la masse de cette boule. Par analogie avec une discrétisation explicite en temps (qui nécessite 3 points en temps pour approcher la dérivée seconde), on obtiendrait

$$u^{n+1} = u^n - \mu\, J'(u^n) + \nu(u^n - u^{n-1}), \tag{3.29}$$

où $\mu > 0$ et $\nu > 0$ sont deux paramètres positifs. Bien sûr, il faut joindre à cette récurrence pour $n \geq 1$ deux initialisations $u^1, u^0 \in V$. Pour simplifier l'analyse on se place en dimension finie.

Théorème 3.3.15 *On suppose que J est convexe, de classe C^2 sur $V = \mathbb{R}^N$, tel que, pour $0 < \alpha \leq L$,*

$$\alpha\,\mathrm{Id} \leq J''(u) \leq L\,\mathrm{Id}.$$

Alors, si u^1, u^0 sont suffisamment proches de l'unique point de minimum u de J, pour $0 \leq \nu < 1$ et $0 < \mu < 2(1+\nu)/L$, l'algorithme de la boule pesante converge et, pour $\nu = (\frac{\sqrt{L}-\sqrt{\alpha}}{\sqrt{L}+\sqrt{\alpha}})^2$ et $\mu = 4/(\sqrt{L}+\sqrt{\alpha})^2$, il existe $C > 0$ tel que

$$\|u^n - u\| \leq C\gamma_{\mathrm{BP}}^n \|u^0 - u\| \quad \text{avec} \quad \gamma_{\mathrm{BP}} = \frac{\sqrt{L}-\sqrt{\alpha}}{\sqrt{L}+\sqrt{\alpha}}.$$

Démonstration. On utilise un argument de stabilité locale, très similaire à celui qui permet d'analyser la convergence de l'algorithme de Newton. Pour cette raison, on suppose qu'à partir d'un certain rang toutes les itérées u^n sont proches de la solution u. Un développement de Taylor donne

$$J'(u^n) = J''(u)(u^n - u) + o(\|u^n - u\|),$$

c'est-à-dire que

$$u^{n+1} = u^n - \mu\, J''(u)(u^n - u) + \nu(u^n - u^{n-1}) + o(\|u^n - u\|).$$

Soit z^n le vecteur $(u^n - u, u^{n-1} - u)^*$ et $H = J''(u)$. On a

$$z^{n+1} = Az^n + o(z^n) \quad \text{avec} \quad A = \begin{pmatrix} (1+\nu)\,\mathrm{Id} - \mu H & -\nu\,\mathrm{Id} \\ \mathrm{Id} & 0 \end{pmatrix}.$$

Pour étudier la convergence de la suite z^n on analyse les valeurs propres ρ de A qui vérifient, pour $(x, y) \neq 0$,

$$A \begin{pmatrix} x \\ y \end{pmatrix} = \rho \begin{pmatrix} x \\ y \end{pmatrix}. \tag{3.30}$$

En effet, si toutes les valeurs propres vérifient $|\rho| \leq \rho_{\max} < 1$, alors un lemme classique d'algèbre linéaire (voir la proposition 13.1.7 dans [1]) affirme que, pout tout $\delta > 0$ (aussi petit que l'on veut), il existe une norme matricielle (subordonnée à une norme vectorielle) telle que $\|A\| \leq \rho_{\max} + \delta$. On déduit alors de $z^{n+1} = Az^n + o(z^n)$ que, pour n suffisamment grand, $\|o(z^n)\| \leq \delta\|z^n\|$ et

$$\|z^{n+1}\| \leq (\rho_{\max} + 2\delta)\|z^n\|,$$

d'où l'on déduit la convergence de z^n vers 0 (et aussi le fait que si u^n est proche de u à un certain rang, cela restera vrai pour les itérations suivantes).

Or ρ est valeur propre dans (3.30) si $Hx = \lambda x$ avec $\lambda = (1 + \nu - \rho - \nu/\rho)/\mu$ où $\lambda \in [\alpha, L]$ est une valeur propre de la Hessienne H. Autrement dit, ρ est une racine du polynome

$$\rho^2 - (1 + \nu - \mu\lambda)\rho + \nu = 0,$$

dont le discriminant est $\Delta = (1+\nu-\mu\lambda)^2 - 4\nu$. On cherche des conditions suffisantes pour que les racines vérifient $|\rho| < 1$. On choisit $\nu \geq 0$ et comme ν est le produit des racines du polynome, il faut que $\nu < 1$. Si $\Delta \leq 0$, alors les deux racines sont complexes et de module égal à $\sqrt{\nu} < 1$. Si $\Delta > 0$, alors un calcul facile montre que les deux racines appartiennent à l'intervalle $(-1, +1)$ si et seulement si $0 < \mu < 2(1+\nu)/\lambda$ pour toute valeur propre λ de H. Par conséquent, les conditions $0 < \nu < 1$ et $0 < \mu < 2(1 + \nu)/L$ assurent la convergence de l'algorithme. Pour trouver des valeurs optimales des paramètres, une analyse (un peu longue et pas détaillée ici) montre qu'il est bon de se placer dans le cas des racines complexes de module égal à $\sqrt{\nu}$, ce qui arrive lorsque $\Delta \leq 0$. On vérifie que $\Delta \leq 0$ est équivalent à

$$(1 - \sqrt{\nu})^2 \leq \mu\lambda \leq (1 + \sqrt{\nu})^2,$$

qui doit être vrai pour toute valeur propre $\lambda \in [\alpha, L]$ de H. C'est donc vrai en particulier si

$$\sqrt{\nu} \geq \sqrt{\alpha\mu} - 1, \quad \sqrt{\nu} \geq 1 - \sqrt{L\mu}.$$

On choisit $\sqrt{\mu} = 2/(\sqrt{\alpha}+\sqrt{L})$ pour que les deux bornes inférieures de $\sqrt{\nu}$ coincident et on prend la plus petite valeur qui en résulte, c'est-à-dire $\sqrt{\nu} = \frac{\sqrt{\alpha}-\sqrt{L}}{\sqrt{\alpha}+\sqrt{L}}$. □

Remarque 3.3.16 La vitesse de convergence du Théorème 3.3.15 est bien meilleure que celle de l'algorithme du gradient à pas fixe (voir la Proposition 3.3.1) et un peu

meilleure, mais asymptotiquement comparable (pour α/L petit) à celle de l'algorithme de Nesterov (voir la Proposition 3.3.10) qui s'écrit

$$\gamma_{\text{Nest}} = \left(1 - \sqrt{\frac{\alpha}{L}}\right) > \gamma_{\text{BP}} = \frac{1 - \sqrt{\alpha/L}}{1 + \sqrt{\alpha/L}}.$$

On voit donc, comme $0 < \alpha/L < 1$, que la méthode de la boule pesante avec ces paramètres optimaux converge plus vite que les algorithmes précédents. Malheureusement, les valeurs de α et L ne sont souvent pas connues en pratique et il est difficile d'ajuster les valeurs optimales des paramètres μ et ν. •

3.3.4 Algorithme du gradient conjugué

La méthode du gradient conjugué est une méthode du premier ordre où la direction de descente dans (3.2) n'est pas le gradient mais une combinaison du gradient et de la direction de descente précédente (c'est donc encore une méthode multi-pas). On introduit donc une suite supplémentaire $p^n \in V$ pour la direction de descente. L'algorithme du gradient conjugué s'écrit

$$\left\{ \begin{array}{l} u^{n+1} = u^n - \mu^n\, p^n \\ p^{n+1} = J'(u^{n+1}) + \beta^n p^n \end{array} \right. \tag{3.31}$$

où $\mu^n \in \mathbb{R}$ est le pas optimal qui minimise $\mu \to J(u^n - \mu\, p^n)$, et $\beta^n \in \mathbb{R}$ est un coefficient que l'on peut calculer suivant deux formules différentes. La première formule est dite de Polak-Ribière

$$\beta^n = \frac{\langle J'(u^{n+1}), J'(u^{n+1}) - J'(u^n)\rangle}{\|J'(u^n)\|^2},$$

tandis que la seconde est appelée Fletcher-Rieves

$$\beta^n = \frac{\|J'(u^{n+1})\|^2}{\|J'(u^n)\|^2}.$$

Expliquons d'où viennent ces formules en quelques mots. Tout d'abord, la recherche linéaire pour trouver le pas μ^n est classique et reminiscente de l'algorithme du gradient à pas optimal. La formule pour β^n s'inspire du cas où $J(u)$ est quadratique. Si $J(u)$ est une fonction non-linéaire quelconque, l'algorithme du gradient conjugué fonctionnera néanmoins, approximativement comme dans le cas quadratique, au voisinage d'un point de minimum non dégénéré (c'est-à-dire où la Hessienne est définie positive). Nous allons donc formellement supposer que la Hessienne $J''(u)$ est indépendante de u et égale à une matrice constante A. L'idée essentielle est de demander à ce que les directions de descente p^n et p^{n+1} soient conjuguées par rapport à la matrice A, c'est-à-dire que $p^{n+1} \cdot Ap^n = 0$. Comme expliqué ci-dessous (voir l'Exercice 3.3.2), cette propriété est à la source de la convergence de l'algorithme du gradient conjugué pour une fonction quadratique et on essaye donc de la reproduire dans le cas général.

On souhaite donc que la valeur de β^n soit telle que

$$p^{n+1} \cdot Ap^n = 0.$$

On remplace p^{n+1} par $J'(u^{n+1}) + \beta^n p^n$ pour obtenir

$$\beta^n = -\frac{J'(u^{n+1}) \cdot Ap^n}{Ap^n \cdot p^n}, \tag{3.32}$$

puis, comme $p^n = (u^n - u^{n+1})/\mu^n$, on a

$$\mu^n Ap^n = Au^n - Au^{n+1} = J'(u^n) - J'(u^{n+1})$$

puisque la Hessienne est constante et égale à A. Or la condition d'optimalité pour μ^n dit que $J'(u^{n+1}) \cdot p^n = 0$ tandis que $J'(u^n) \cdot p^n = J'(u^n) \cdot (J'(u^n) + \beta_{n-1} p^{n-1})$ et $J'(u^n) \cdot p^{n-1} = 0$ à cause de l'optimalité de μ^{n-1}. Donc

$$Ap^n \cdot p^n = \|J'(u^n)\|^2/\mu^n. \tag{3.33}$$

Par ailleurs,

$$J'(u^{n+1}) \cdot Ap^n = J'(u^{n+1}) \cdot (J'(u^n) - J'(u^{n+1}))/\mu^n, \tag{3.34}$$

d'où l'on déduit la formule de Polak-Ribière en reportant (3.33) et (3.34) dans (3.32). Un calcul similaire conduit à la formule de Fletcher-Rieves. Remarquons que ces deux formules ne sont pas égales dans le cas général mais qu'elles coincident pour une fonction $J(u)$ quadratique.

En fait, la méthode du gradient conjugué a d'abord été inventée pour résoudre des systèmes linéaires dont la matrice est symétrique, définie positive, c'est-à-dire pour minimiser sur $\mathbb{R}^N$ une fonction quadratique

$$J(x) = \frac{1}{2} Ax \cdot x - b \cdot x$$

avec $b \in \mathbb{R}^N$ et A une matrice symétrique définie positive. C'est dans ce cas quadratique que l'algorithme du gradient conjugué est le plus efficace puisqu'on peut même démontrer qu'il converge exactement en au plus N itérations ! Dans ce cas quadratique, les formules (3.31) se simplifient en

$$\text{pour } n \geq 0 \quad \begin{cases} x_{n+1} = x_n - \mu_n p_n \\ r_{n+1} = r_n - \mu_n Ap_n \\ p_{n+1} = r_{n+1} + \beta_n p_n \end{cases} \tag{3.35}$$

avec une initialisation $x_0 \in \mathbb{R}^N$, $p_0 = r_0 = Ax_0 - b$ et

$$\mu_n = \frac{\|r_n\|^2}{Ap_n \cdot p_n} \text{ et } \beta_n = \frac{\|r_{n+1}\|^2}{\|r_n\|^2}.$$

On vérifie facilement que μ_n est précisément l'unique point de minimum du polynôme quadratique $\mu \to J(x_n - \mu p_n)$. On peut vérifier par récurrence que la suite r_n coincide avec le gradient $J'(x_n)$ et donc que β_n est donné par la formule de Fletcher-Rieves.

Exercice 3.3.2 Soit A une matrice symétrique définie positive. Soit $x_0 \in \mathbb{R}^N$. Pour la récurrence (3.35) démontrer que $r_n = Ax_n - b$. Montrer que la suite p_n vérifie $Ap_n \cdot p_j = 0$ pour $0 \leq j < n$ (on dit que la suite p_n est conjuguée par rapport à A). Vérifier que les formules de Fletcher-Rieves et Polak-Ribière coincident dans ce cas.

Le fait que la suite des directions de descente p_n soit conjuguée par rapport à A donne son nom à l'algorithme. C'est une propriété cruciale pour l'efficacité de la méthode car elle assure que la nouvelle direction de descente p_n "travaille" de manière orthogonale (au sens du produit scalaire $\langle x, y \rangle_A = Ax \cdot y$) par rapport aux directions précédentes. Le nouveau gain de minimisation ne nuit pas aux gains passés (pas d'effet de convergence en zig-zag) et c'est la raison du résultat de convergence exacte en moins de N itérations. Néanmoins, la convergence approchée peut avoir lieu en beaucoup moins d'itérations comme le montre le résultat suivant que nous admettrons.

Proposition 3.3.17 *Soit A une matrice symétrique réelle définie positive, de valeurs propres ordonnées $0 < \lambda_1 \leq \cdots \leq \lambda_N$. Soit x la solution exacte du système $Ax = b$. Soit $(x_n)_{n \geq 0}$ la suite de solutions approchées du gradient conjugué, définie par* (3.35)*. Alors*

$$\|x_n - x\|_2 \leq 2\sqrt{\lambda_N/\lambda_1} \left(\frac{1 - \sqrt{\lambda_1/\lambda_N}}{1 + \sqrt{\lambda_1/\lambda_N}} \right)^n \|x_0 - x\|_2.$$

La vitesse de convergence de l'algorithme du gradient conjugué est exactement celle de l'algorithme de la boule pesante et donc bien meilleure que celle de l'algorithme du gradient à pas fixe. Mais ce qui est remarquable dans l'algorithme du gradient conjugué, c'est l'absence de paramètres à régler dans la formule de récurrence (3.35). On n'a pas à connaitre les valeurs propres λ_1, λ_N pour obtenir la vitesse de convergence de la Proposition 3.3.17.

Remarque 3.3.18 Le rapport $\lambda_N/\lambda_1 \geq 1$, qui apparait dans la vitesse de convergence de la méthode du gradient conjugué, est appelé le conditionnement de la matrice A. Pour accélérer encore la convergence de la méthode du gradient conjugué, une idée est de **préconditionner** le système linéaire $Ax = b$ en le pré-multipliant par une matrice C^{-1} telle que le conditionnement de $(C^{-1}A)$ soit plus petit que celui de A. En pratique on choisit une matrice C "proche" de A mais plus facile à inverser (voir [1] pour des détails). Cette idée est semblable à celle que nous avons vue à la Remarque 3.2.4. •

3.3.5 Algorithme de sous-gradient

Les méthodes de gradient peuvent se généraliser au cas des fonctions qui ne sont pas différentiables mais sont convexes. Cette généralisation s'appelle la **méthode de sous-gradient**. Il nous faut d'abord définir la notion de **sous-gradient** pour les fonctions convexes. Par souci de simplicité on se limite à la dimension finie mais le cas des espaces de Hilbert n'est pas vraiment plus difficile.

Définition 3.3.19 *Si J est une fonction convexe de $\mathbb{R}^n$ dans $\mathbb{R}$, on appelle* **sous-différentiel** *de J en x l'ensemble*

$$\partial J(x) = \{p \in \mathbb{R}^n \mid J(y) - J(x) \geq p \cdot (y - x), \qquad \forall y \in \mathbb{R}^n\} \ . \tag{3.36}$$

Les éléments de $\partial J(x)$ sont appelés **sous-gradients**.

Il résulte aussitôt de cette définition que le sous-différentiel $\partial J(x)$ est un convexe fermé de $\mathbb{R}^n$. D'autre part, si J est dérivable au point x, on vérifie aisément que le sous-différentiel est un singleton, $\partial J(x) = \{J'(x)\}$.

Lemme 3.3.20 *Soit J une fonction convexe de $\mathbb{R}^n$ dans $\mathbb{R}$. Si $x \in \mathbb{R}^n$ vérifie $0 \in \partial J(x)$, alors x est un point de minimum de J.*

Démonstration. La démonstration est évidente par définition du sous-différentiel. Remarquons que la condition $0 \in \partial J(x)$ généralise au cas convexe non-différentiable la condition d'optimalité usuelle $J'(x) = 0$ (lorsque J est dérivable). □

Exercice 3.3.3 Soit $J(x) = |x|$ définie de $\mathbb{R}$ dans $\mathbb{R}$. Calculer son sous-gradient $\partial J(0)$.

Une classe importante de fonctions convexes, qui ne sont pas différentiables mais admettent un sous-gradient que l'on sait calculer, est celle des fonctions qui sont des maxima de fonctions convexes différentiables. C'est une situation courante puisque, par exemple, on sait, d'après l'Exercice 2.3.2, que toute fonction convexe est le supremum des fonctions affines qui la minorent. Plus généralement, pour un espace (non vide) de paramètres Λ, on définit

$$\mathcal{J}(x) = \sup_{\lambda \in \Lambda} J_\lambda(x), \tag{3.37}$$

où chaque fonction $J_\lambda(x)$ est convexe et différentiable sur $\mathbb{R}^n$. Pour simplifier, on va supposer que l'ensemble des paramètres Λ est fini. On calcul alors aisément un sous-gradient de $\mathcal{J}$.

Lemme 3.3.21 *Supposons Λ fini, soit $\mathcal{J}$ la fonction définie par* (3.37) *avec des fonctions $J_\lambda(x)$ convexes et différentiables, et pour tout $x \in \mathbb{R}^n$, posons $\Gamma(x) = \{\lambda \in \Lambda \mid J_\lambda(x) = \mathcal{J}(x)\}$. Alors, pour tout $\lambda \in \Gamma(x)$, $J'_\lambda(x)$ est un sous-gradient de $\mathcal{J}$ au point x.*

Démonstration. Pour tout $\lambda \in \Gamma(x)$, et pour tout $y \in \mathbb{R}^n$, on a $\mathcal{J}(y) - \mathcal{J}(x) \geq J_\lambda(y) - J_\lambda(x) \geq J'_\lambda(x) \cdot (y - x)$, par convexité de J_λ, ce qui démontre le résultat. □

Remarque 3.3.22 Les fonctions non régulières du type (3.37) se retrouvent dans au moins deux contextes importants. D'une part, elles sont utilisées dans les méthodes de relaxations Lagrangiennes pour l'optimisation combinatoire ou en variables entières (voir [5]). D'autre part, elles correspondent à l'approche, dite **du pire des cas**, en optimisation robuste. Supposons que l'on veuille optimiser un système décrit par une variable x. Mais la description de ce système est entachée d'incertitudes, d'erreurs ou de données inconnues, caractérisées par λ. On peut identifier chaque

valeur de λ à un scenario possible de fonctionnement du système. Si l'on veut optimiser pour tous les scenarios possibles, l'approche du pire des cas consiste à minimiser le maximum par rapport à la variable λ. Notons que c'est une approche assez pessimiste (on prévoit le pire !) et qu'elle est parfois remplacée par une approche **en moyenne** où on optimise la moyenne de $J_\lambda(x)$ sur Λ (ou bien une somme pondérée de la moyenne et de la variance). •

L'algorithme de sous-gradient pour minimiser la fonction convexe $\mathcal{J}$ consiste, pour une initialisation $x_0 \in \mathbb{R}^n$, à construire la suite

$$x_{k+1} = x_k - \frac{\rho_k}{\|p_k\|} p_k \ , \tag{3.38}$$

où p_k est un sous-gradient quelconque de $\mathcal{J}$ au point x_k, et où $\rho_k > 0$ est une suite de réels strictement positifs telle que

$$\rho_k \to 0, \quad \sum_{i\in\mathbb{N}} \rho_i = +\infty, \quad \sum_{i\in\mathbb{N}} \rho_i^2 < +\infty \, . \tag{3.39}$$

évidemment, la valeur x_{k+1} n'est bien définie que si $p_k \neq 0$. Lorsque $p_k = 0$, l'algorithme s'arrête : x_k est alors le minimum de $\mathcal{J}$ en vertu du Lemme 3.3.20. Un exemple de suite de pas ρ_k vérifiant (3.39) est $\rho_k = 1/(k+1)^{1/2+\epsilon}$ avec $\epsilon > 0$.

Proposition 3.3.23 *Soit une fonction $\mathcal{J}$ convexe, infinie à l'infini, admettant en tout point un sous-différentiel non-vide et localement Lipschitzienne, au sens où, pour tout $M > 0$, il existe $L_M > 0$ tel que*

$$\|x\| + \|y\| \leq M \Rightarrow \|\mathcal{J}(x) - \mathcal{J}(y)\| \leq L_M \|x - y\|.$$

Alors, si les pas vérifient (3.39)*, l'algorithme du sous-gradient converge.*

Remarque 3.3.24 Les hypothèses de la Proposition 3.3.23 sont vérifiées pour les fonctions $\mathcal{J}$ définies par (3.37) avec un ensemble Λ fini, chacune des fonctions $J_\lambda(x)$ étant convexe différentiable et l'une d'entre elle infinie à l'infini. •

Démonstration. Comme $\mathcal{J}$ est continue et infinie à l'infini, elle admet au moins un point de minimum x^*. En utilisant (3.38) on développe la norme suivante

$$\begin{aligned} \|x_{k+1} - x_*\|^2 &= \|x_k - x_*\|^2 - 2\frac{\rho_k}{\|p_k\|} p_k \cdot (x_k - x_*) + \rho_k^2 \\ &\leq \|x_k - x_*\|^2 - 2\frac{\rho_k}{\|p_k\|} \left(\mathcal{J}(x_k) - \mathcal{J}(x_*)\right) + \rho_k^2 \end{aligned} \tag{3.40}$$

car p_k est un sous gradient de $\mathcal{J}$ au point x_k. On somme ces inégalités et une minoration évidente conduit à

$$\|x_{i+1} - x_*\|^2 + 2 \min_{0\leq k\leq i} \left(\mathcal{J}(x_k) - \mathcal{J}(x_*)\right) \sum_{k=0}^{i} \frac{\rho_k}{\|p_k\|} \leq \|x_0 - x_*\|^2 + \sum_{k=0}^{i} \rho_k^2. \tag{3.41}$$

On en déduit tout d'abord que $\|x_{i+1}-x_*\|$ est bornée à cause de la dernière condition de (3.39), donc la suite x_k est bornée par une constante M indépendante de k. On applique alors l'hypothèse que $\mathcal{J}$ est Lipschitzienne à la définition du sous-gradient p_k

$$p_k \cdot (y - x_k) \leq J(y) - J(x_k) \leq L_M \|y - x_k\|$$

d'où l'on déduit en variant y autour de x_k que $\|p_k\| \leq L_M$. Par conséquent (3.41) implique que

$$\min_{0\leq k\leq i} (\mathcal{J}(x_k) - \mathcal{J}(x_*)) \leq \frac{L_M}{2} \frac{\|x_0 - x_*\|^2 + \sum_{k=0}^{\infty} \rho_k^2}{\sum_{k=0}^{i} \rho_k},$$

ce qui démontre que $\liminf_{k\to+\infty} \mathcal{J}(x_k) = \mathcal{J}(x_*)$ puisque la série des ρ_k diverge. Soit $x_{k'}$ une sous-suite de x_k telle que $\lim_{k\to+\infty} \mathcal{J}(x_{k'}) = \mathcal{J}(x_*)$. Comme elle est bornée dans $\mathbb{R}^n$, il existe encore une sous-suite, toujours notée $x_{k'}$ pour souci de simplicité dans les notations, qui converge vers une limite x'_* qui, par continuité de J est un point de minimum de J. Cette limite x'_* n'est peut-être pas le point de minimum x_* introduit ci-dessus mais rien ne nous empêche de remplacer x_* par x'_* et tout le raisonnement reste le même car les itérées x_k et p_k ne dépendent pas du choix x_* ou x'_*. Par souci de simplicité dans les notations, on réécrit $x'_* = x_*$. Montrons que toute la suite x_k converge en fait vers x_*.

Pour $\delta > 0$ petit fixé, il existe un indice k', suffisamment grand, tel que

$$\|x_{k'} - x_*\|^2 \leq \delta/2 \quad \text{et} \quad \sum_{i\geq k'} \rho_i^2 \leq \delta/2.$$

Alors, pour tout indice $k \geq k'$, comme (3.40) implique que

$$\|x_{k+1} - x_*\|^2 \leq \|x_k - x_*\|^2 + \rho_k^2,$$

en sommant on obtient

$$\|x_{k+1} - x_*\|^2 \leq \|x_{k'} - x_*\|^2 + \sum_{i=k'}^{k} \rho_i^2 \leq \delta,$$

ce qui démontre que toute la suite x_k converge vers le point de minimum x_*. □

Remarque 3.3.25 La vitesse de convergence de l'algorithme de sous-gradient (3.38) est très lente puisque pour $\rho_k = 1/(k+1)^{1/2+\epsilon}$ avec $\epsilon > 0$ on trouve que

$$0 \leq \min_{0\leq k\leq i} \mathcal{J}(x_k) - \mathcal{J}(x_*) \leq \frac{C}{i^{1/2-\epsilon}},$$

ce qui est bien plus lent que la convergence géométrique pour l'algorithme du gradient (voir la Remarque 3.2.8). Si on suppose une propriété du type α-convexité pour J en x_*, alors on obtient la même vitesse de convergence pour $\|x_k - x_*\|^2$. •

Remarque 3.3.26 L'algorithme de sous-gradient (3.38) ne garantit pas la décroissance de la fonction objectif aux points où celle-ci n'est pas différentiable. En effet, considérons l'exemple $\mathcal{J}(x) = \max(J_1(x), J_2(x))$ avec J_1, J_2 convexes différentiables, plaçons nous en un point x où $J_1(x) = J_2(x)$ et prenons le sous-gradient $p = J_1'(x) \neq 0$. Dès que $-J_1'(x)$ n'est pas une direction de descente pour J_2, pour tout pas $\rho > 0$ suffisamment petit, on obtient $\mathcal{J}(x - \rho p/\|p\|) = J_2(x - \rho p/\|p\|) > J_2(x) = \mathcal{J}(x)$. •

Remarque 3.3.27 Dans l'algorithme de sous-gradient (3.38) on normalise le sous-gradient p_k. En effet, comme l'a montré l'exemple de l'Exercice 3.3.3, la norme d'un sous-gradient peut-être très variable et la valeur de cette norme ne donne aucun indication sur l'optimalité ou non du point où on l'a calculé. •

3.3.6 Gradient stochastique

Il s'agit d'un algorithme qui est dédié à une classe particulière de problèmes d'optimisation, très utile pour l'apprentissage machine, dont une situation typique est présentée dans l'Exemple 1.2.6. Dans ce contexte, il est naturel de se restreindre à la dimension finie $V = \mathbb{R}^d$. La spécificité de ce type de problèmes est que la fonction à minimiser est une somme (ou une moyenne) d'un très grand nombre n de fonctions. Par exemple, en apprentissage machine n est le nombre de données à partir desquelles on veut apprendre les paramètres $x \in \mathbb{R}^d$ du modèle explicatif. Autrement dit, étant données n fonctions f_i, on considère

$$\inf_{x\in\mathbb{R}^d} F(x) = \frac{1}{n}\sum_{i=1}^{n} f_i(x). \tag{3.42}$$

Rappelons que l'algorithme du gradient "classique" (qu'on appelle dans ce contexte, algorithme de *batch*) construit la suite

$$x^{k+1} = x^k - \frac{\mu^k}{n}\sum_{i=1}^{n} f_i'(x^k)$$

où $\mu^k > 0$ est un pas de descente (fixe ou optimal). Par contraste, **l'algorithme du gradient stochastique** n'utilise qu'une seule dérivée de fonction f_i par itération. Autrement dit, à partir d'une initialisation $x^0 \in \mathbb{R}^d$, on construit la suite

$$x^{k+1} = x^k - \mu^k f_{i_k}'(x^k), \tag{3.43}$$

où $\mu^k > 0$ est un pas de descente et i_k est un indice tiré aléatoirement (indépendamment des précédents tirages) et uniformément dans l'ensemble $\{1, ..., n\}$.

Proposition 3.3.28 *On suppose que $F(x)$ est α-convexe et qu'il existe une constante C, indépendante de n, telle que pour tout $x \in \mathbb{R}^d$*

$$\frac{1}{n}\sum_{i=1}^{n} \|f_i'(x)\|^2 \leq C(1 + \|x - x^*\|^2) \tag{3.44}$$

où x^* désigne l'unique point de minimum de $F(x)$. Si la suite des pas vérifie

$$\mu^k = \frac{1}{k+1},$$

alors l'algorithme du gradient stochastique (3.43) *converge (en moyenne).*

Remarque 3.3.29 Les hypothèses sur F sont vérifiées dans le cas de l'Exemple 1.2.6 (car chacune des dérivées $f_i'(x)$ est bornée) ou bien si chaque fonction f_i est quadratique et fortement convexe, uniformément par rapport à i. •

Démonstration. On réécrit (3.43) sous la forme

$$x^{k+1} - x^* = x^k - x^* - \mu^k f_{i_k}'(x^k),$$

dont on calcule la norme au carré

$$\|x^{k+1} - x^*\|^2 = \|x^k - x^*\|^2 - 2\mu^k (x^k - x^*) \cdot f_{i_k}'(x^k) + (\mu^k)^2 \|f_{i_k}'(x^k)\|^2. \tag{3.45}$$

On prend l'espérance de (3.45) par rapport à la seule variable aléatoire qui gouverne le choix de l'indice i_k, en notant que x^k ne dépend pas de cette variable aléatoire, pour obtenir

$$\mathbb{E}\Big(\|x^{k+1} - x^*\|^2\Big) = \|x^k - x^*\|^2 - 2\mu^k (x^k - x^*) \cdot F'(x^k) + \frac{(\mu^k)^2}{n} \sum_{i=1}^n \|f_i'(x^k)\|^2$$

car, par l'hypothèse de tirage uniforme de i_k, on a

$$\mathbb{E}\Big(f_{i_k}'(x^k)\Big) = \frac{1}{n}\sum_{i=1}^n f_i'(x^k) = F'(x^k) \quad \text{et} \quad \mathbb{E}\Big(\|f_{i_k}'(x^k)\|^2\Big) = \frac{1}{n}\sum_{i=1}^n \|f_i'(x^k)\|^2.$$

Par la forte convexité de F, voir (2.24), on en déduit

$$\mathbb{E}\Big(\|x^{k+1} - x^*\|^2\Big) \le \big(1 - 2\mu^k\alpha\big)\, \|x^k - x^*\|^2 + \frac{(\mu^k)^2}{n} \sum_{i=1}^n \|f_i'(x^k)\|^2.$$

Utilisant l'hypothèse (3.44), il vient

$$\mathbb{E}\Big(\|x^{k+1} - x^*\|^2\Big) \le \big(1 - 2\mu^k\alpha + C(\mu^k)^2\big)\, \|x^k - x^*\|^2 + C(\mu^k)^2.$$

On choisit le pas de descente $0 < \mu^k < 2\alpha/C$ de manière à ce que le coefficient d'amplification ρ^k soit strictement plus petit que 1

$$0 < \rho^k = 1 - 2\mu^k\alpha + C(\mu^k)^2 < 1.$$

Une récurrence facile montre alors que

$$\mathbb{E}\Big(\|x^{k+1} - x^*\|^2\Big) \le \Pi^k \|x^0 - x^*\|^2 + C\Pi^k \sum_{i=0}^k \frac{(\mu^i)^2}{\Pi^i}, \tag{3.46}$$

où, cette fois-ci, $\mathbb{E}$ désigne l'espérance pour toutes les variables aléatoires des indices de i_0 à i_k, et avec

$$\Pi^k = \prod_{j=0}^{k} \rho^j.$$

On voit immédiatement que pour démontrer la convergence de l'algorithme, non seulement le pas μ^k ne doit pas être trop grand pour que Π^k tende vers zéro mais il doit aussi lui-même tendre vers zéro pour que la série dans (3.46) converge. C'est une situation similaire à celle de l'algorithme du sous-gradient (voir la Proposition 3.3.23). On vérifie que, si $0 < \alpha < 1/2$, le choix $\mu^k = 1/(k+1)$ conduit à

$$\rho^k = 1 - \frac{2\alpha}{k+1} + \mathcal{O}(k^{-2}), \quad \Pi^k = \mathcal{O}(k^{-2\alpha}), \quad \Pi^k \sum_{i=0}^{k} \frac{(\mu^i)^2}{\Pi^i} = \mathcal{O}(k^{-2\alpha}),$$

ce qui prouve la convergence en moyenne (un résultat similaire avec d'autres exposants s'obtient lorsque $\alpha \geq 1/2$). On peut démontrer la convergence presque sûrement mais cela dépasse la cadre de ce cours. □

Comme pour l'algorithme du sous-gradient la convergence est particulièrement lente (algébrique au lieu de géométrique pour le gradient à pas fixe). Du coup, il n'est pas clair que cet algorithme soit préférable à celui du gradient. Mais il ne faut pas oublier que si n est très grand, le coût d'une itération de gradient stochastique est approximativement n plus fois faible que le coût d'une itération du gradient puisqu'on n'évalue qu'une seule dérivée f_i' au lieu de n (pour les deux algorithmes on calcule aussi régulièrement la fonction objectif F pour vérifier sa décroissance et la bonne convergence des itérations). Par ailleurs, il existe des stratégies heuristiques du choix du pas de descente μ^k qui peuvent améliorer en pratique le choix théorique ci-dessus. En particulier, l'algorithme du gradient stochastique est souvent plus rapide lors des premières itérations que l'algorithme du gradient et surtout plus insensible au "bruit" (qu'il soit numérique ou dans les données).

Remarque 3.3.30 Entre l'algorithme du gradient stochastique (3.43) et l'algorithme classique du gradient (dit "batch") on peut proposer un algorithme de **mini-batch** qui sélectionne une collection de $m \geq 1$ indices

$$x^{k+1} = x^k - \mu^k \frac{1}{m} \sum_{i \in \mathcal{I}_k} f_i'(x^k), \tag{3.47}$$

où $\mu^k > 0$ est un pas de descente et $\mathcal{I}_k$ est une collection de m indices distincts tirés aléatoirement (indépendamment des précédents tirages) et uniformément dans l'ensemble $\{1, ..., n\}$. •

3.3.7 Algorithme proximal

Pour motiver ce nouvel algorithme nous rappelons l'Exemple 1.2.4 de problème aux moindres carrés avec régularisation ℓ^1 pour obtenir des solutions creuses ou

parcimonieuses. Soit $b \in \mathbb{R}^p$ et A une matrice de taille $p \times n$, Pour un paramètre $\tau > 0$, on cherche une solution $x \in \mathbb{R}^n$ du problème de minimisation

$$\min_{x\in\mathbb{R}^n} \left\{ \frac{1}{2\tau}\|Ax-b\|^2 + \|x\|_1 \right\} \quad \text{avec} \quad \|x\|_1 = \sum_{j=1}^{n} |x_j|. \tag{3.48}$$

Plus τ est grand et plus la solution x aura des composantes nulles. Au contraire, à la limite $\tau \to 0^+$, il s'agit d'un problème de moindres carrés standard dont on sait qu'il admet toujours au moins une solution qui est unique si $\mathrm{Ker}A = \{0\}$ (cf. Exercice 2.5.2).

Remarque 3.3.31 Expliquons sur un cas particulier très simple pourquoi la norme ℓ^1 dans (3.48) conduit à ce que la solution ait de nombreuses composantes nulles. On choisit $p = n$ et $A = \mathrm{Id}$. Dans ce cas, la minimisation de (3.48) se fait "à la main", composante par composante. La fonction de $\mathbb{R}$ dans $\mathbb{R}$,

$$x_i \ \to \ \frac{1}{2\tau}|x_i - b_i|^2 + |x_i|,$$

est fortement convexe, dérivable partout sauf en 0, et admet donc un unique point de minimum x_i^* qui vérifie, soit $x_i^* = 0$, soit la condition d'optimalité

$$x_i^* - b_i + \tau = 0 \text{ si } x_i^* > 0, \quad x_i^* - b_i - \tau = 0 \text{ si } x_i^* < 0.$$

On en déduit facilement l'unique point de minimum qui est donné par l'opérateur de **contraction** (shrinkage, en anglais)

$$x_i^* = S_\tau(b_i) = \begin{cases} 0 & \text{si } |b_i| < \tau, \\ b_i - \ \mathrm{sgn}(b_i)\tau & \text{si } |b_i| \geq \tau. \end{cases} \tag{3.49}$$

L'interprétation est simple si l'on pense que la donnée b est bruitée et que τ correspond au seuil maximal du bruit. Si $|b_i| < \tau$, la donnée est indistinguable du bruit et on obtient $x_i^* = 0$. Sinon, on obtient $x_i^* \approx b_i$ à une correction de l'ordre de τ près. La solution $x^* \in \mathbb{R}^n$ est donc d'autant plus parcimonieuse que le seuil τ est élevé. •

Pour résoudre (3.48) on ne peut pas employer une méthode de gradient classique car la norme ℓ^1, $x \to \|x\|_1$, n'est pas dérivable en tout point dont une des composantes s'annule. On pourrait utiliser l'algorithme du sous-gradient car la fonction objectif est convexe mais on a vu que cet algorithme converge lentement. Il y a donc de la place pour concevoir un algorithme plus efficace. Plus généralement, on considère la minimisation de la somme de deux fonctions convexes sur $\mathbb{R}^n$

$$\min_{x\in\mathbb{R}^n} J_1(x) + J_2(x)$$

avec J_1 différentiable et J_2 convexe mais non différentiable. L'exemple ci-dessus correspond à $J_1(x) = \frac{1}{2\tau}\|Ax-b\|^2$ et $J_2(x) = \|x\|_1$. L'idée est d'appliquer deux algorithmes différents à J_1 et J_2. Evidemment, pour J_1 on a l'embarras du choix car nous avons déjà vu plein d'algorithmes de type gradient ! Par contre, pour J_2

nous ne connaissons que l'algorithme de sous-gradient dont on sait qu'il est peu efficace. C'est pourquoi nous introduisons maintenant l'algorithme **proximal** dont le principe est de faire un algorithme de gradient **implicite**, c'est-à-dire, pour un pas de descente $\tau > 0$, de construire la suite des itérées

$$x^{k+1} = x^k - \tau J_2'(x^{k+1}). \tag{3.50}$$

Evidemment, c'est un algorithme "conceptuel" car il faut résoudre une équation implicite en x^{k+1}, ce qui n'est pas facile. En plus, il faudrait que J_2 soit différentiable, ce qui n'est pas le cas. Néanmoins, on remarque que (3.50) serait la condition d'optimalité de

$$\min_{x\in\mathbb{R}^n} \frac{1}{2\tau}\|x - x^k\|^2 + J_2(x) \tag{3.51}$$

si x^{k+1} en est le point de minimum. Or, si (3.50) n'a pas de sens pour une fonction J_2 non différentiable, au contraire (3.51) admet bien un unique point de minimum, noté x^{k+1}, dès que J_2 est convexe, puisque la fonction à minimiser est fortement convexe. Ainsi, (3.51) permet de définir **l'application proximale** de τJ_2

$$\mathrm{prox}_{\tau J_2}(x^k) = x^{k+1}. \tag{3.52}$$

Avec une initialisation $x^0 \in \mathbb{R}^n$, (3.52) définit un algorithme de minimisation pour J_2, appelé **algorithme proximal**. On vérifie bien qu'il s'agit d'un algorithme de descente car $J_2(x^{k+1}) < J_2(x^k)$, sauf si $x^{k+1} = x^k$ auquel cas la suite x^m est stationnaire pour $m \geq k$. Cependant, cet algorithme est toujours de type conceptuel car pour une fonction convexe quelconque J_2 on ne sait pas résoudre le problème de minimisation (3.51) dont la solution est requise pour définir l'algorithme. En pratique, cet algorithme est donc limité à des fonctions simples pour lesquelles on sait résoudre "à la main" (3.51). C'est précisément le cas pour $J_2(x) = \|x\|_1$, comme on vient de le voir à la Remarque 3.3.31. Dans ce cas, on a $x^{k+1} = \mathrm{prox}_{\tau\|\cdot\|_1}(x^k) = S_\tau(x^k)$, où S_τ est l'opérateur de contraction défini par (3.49).

Finalement, on propose un algorithme de type explicite-implicite pour minimiser $J_1 + J_2$: soit une initialisation $x^0 \in \mathbb{R}^n$, pour $k \geq 0$, on calcule

$$\begin{cases} x^{k+1/2} = x^k - \tau J_1'(x^k), \\ x^{k+1} = \mathrm{prox}_{\tau J_2}(x^{k+1/2}), \end{cases} \tag{3.53}$$

où on a appliqué une itération de l'algorithme du gradient à pas fixe à J_1 suivi d'une itération de l'algorithme proximal à J_2 avec le même pas fixe τ pour les deux. Pour le cas particulier (3.48), cet algorithme proximal est

$$x^{k+1} = S_\tau\Big(x^k - \tau A^*(Ax^k - b)\Big).$$

Exercice 3.3.4 Soit K un convexe fermé de $\mathbb{R}^n$. On définit la fonction indicatrice de K comme l'application de $\mathbb{R}^n$ dans $\mathbb{R} \cup \{+\infty\}$ définie par

$$I_K(x) = \begin{cases} 0 & \text{si } x \in K, \\ +\infty & \text{si } x \notin K. \end{cases}$$

Vérifier que I_K est une fonction convexe et que l'application proximale prox_{I_K} est simplement l'opérateur de projection orthogonale sur K.

Exercice 3.3.5 Soit J une fonction convexe minorée par une fonction affine sur $\mathbb{R}^n$. Pour $\tau > 0$, on définit la régularisation de Moreau-Yosida de J, notée J_τ, par

$$J_\tau(x) = \min_{y\in\mathbb{R}^n} \frac{1}{2\tau}\|x-y\|^2 + J(y).$$

Montrer que J_τ est convexe, différentiable et que $\min_{x\in\mathbb{R}^n} J(x) = \min_{x\in\mathbb{R}^n} J_\tau(x)$ (indication : on pourra utiliser la forte convexité de la fonction dont la minimisation donne J_τ pour montrer d'abord que l'application proximale est différentiable). Vérifier que

$$\text{prox}_{\tau J}(x^k) = x^{k+1} \Leftrightarrow x^{k+1} = x^k - \tau J'_\tau(x^k),$$

c'est-à-dire qu'une itération de l'algorithme proximal pour J est équivalent à une itération de l'algorithme du gradient à pas fixe pour J_τ.

Exercice 3.3.6 Soit J_1 une fonction fortement convexe différentiable sur $\mathbb{R}^n$. Soit K un convexe fermé de $\mathbb{R}^n$ et $J_2 = I_K$, la fonction indicatrice de K définie comme à l'Exercice 3.3.4. Vérifier que l'algorithme explicite-implicite (3.53) n'est rien d'autre que l'algorithme de gradient projeté de la Sous-section 3.4.1.

Exercice 3.3.7 Soit A une matrice $n\times n$ symétrique réelle, définie positive, et $b \in \mathbb{R}^n$. On définit la fonction quadratique J sur $\mathbb{R}^n$ par

$$J(x) = \frac{1}{2}Ax\cdot x - b\cdot x.$$

Déterminer l'opérateur proximal prox_J.

Lemme 3.3.32 *Soit $J(x)$ une fonction convexe différentiable de $\mathbb{R}^n$ dans $\mathbb{R}$ qui admet un unique point de minimum x^*. Pour $\tau > 0$ et $x^0 \in \mathbb{R}^n$ on définit l'algorithme proximal par la suite, indexée par $k \geq 0$,*

$$x^{k+1} = \text{prox}_{\tau J}(x^k) = \arg\min_{x\in\mathbb{R}^n} \left\{ \frac{1}{2\tau}\|x-x^k\|^2 + J(x) \right\}.$$

Alors la suite x^k converge vers le point de minimum x^*. De plus, si J est α-convexe, alors

$$\|x^k - x^*\|^2 \leq \|x^0 - x^*\|^2 \frac{1}{(1+2\tau\alpha)^k}.$$

Remarque 3.3.33 Le Lemme 3.3.32 donne la convergence de l'algorithme proximal sans aucune condition de petitesse sur le pas τ. On retrouve là une caractéristique des algorithmes implicites par rapport aux algorithmes explicites (voir par exemple [1] pour les schémas de différences finies). Par souci de simplicité de sa preuve le Lemme 3.3.32 fait l'hypothèse que J est différentiable mais le résultat s'étend au cas de fonctions qui ne sont pas différentiables grâce à la notion de sous-gradient. •

Démonstration. Comme J est convexe, la fonction $x \to J(x) + \|x - x^k\|^2/(2\tau)$ est $(1/\tau)$-convexe. On écrit sa forte convexité (2.23) à son point de minimum x^{k+1} (où sa dérivée s'annule)

$$J(x) + \frac{1}{2\tau}\|x - x^k\|^2 \geq J(x^{k+1}) + \frac{1}{2\tau}\|x^{k+1} - x^k\|^2 + \frac{1}{2\tau}\|x - x^{k+1}\|^2. \tag{3.54}$$

En prenant $x = x^k$ on trouve que $J(x^{k+1}) \leq J(x^k)$. Si on choisit $x = x^*$ et que l'on somme l'inégalité ci-dessus pour tous les indices de 0 à $(k-1)$, on obtient

$$\frac{1}{2\tau}\|x^* - x^0\|^2 \geq \sum_{i=1}^{k}(J(x^i) - J(x^*)) + \frac{1}{2\tau}\|x^* - x^k\|^2 \geq k(J(x^k) - J(x^*))$$

dont on déduit que $J(x^k)$ converge vers $J(x^*)$ et que la suite (x^k) est bornée dans $\mathbb{R}^n$. Par conséquent, pour une sous-suite, $x^{k'}$ converge vers une limite x^∞ et, comme J est continue et que son point de minimum est unique, il vient que $x^\infty = x^*$ et que toute la suite x^k converge vers x^*.

Si J est α-convexe, alors on peut améliorer (3.54) car $x \to J(x) + \|x - x^k\|^2/(2\tau)$ est $(\alpha + 1/\tau)$-convexe

$$J(x) + \frac{1}{2\tau}\|x - x^k\|^2 \geq J(x^{k+1}) + \frac{1}{2\tau}\|x^{k+1} - x^k\|^2 + \frac{1}{2}(\alpha + 1/\tau)\|x - x^{k+1}\|^2.$$

On choisit encore $x = x^*$ et on combine cette inégalité avec la forte convexité (2.23) de J à son point de minimum x^* (où sa dérivée s'annule)

$$J(x^{k+1}) \geq J(x^*) + \frac{\alpha}{2}\|x^{k+1} - x^*\|^2.$$

En sommant les deux dernières inégalités on obtient

$$\|x^* - x^k\|^2 \geq (1 + 2\tau\alpha)\|x^* - x^{k+1}\|^2$$

d'où le résultat. □

3.4 Algorithmes de type gradient (avec contraintes)

On s'intéresse maintenant à la résolution numérique de problèmes d'optimisation avec contraintes

$$\inf_{v \in K} J(v)\ , \tag{3.55}$$

où J est une fonction α-convexe différentiable définie sur K, sous-ensemble fermé non vide de l'espace de Hilbert réel V. Nous commencerons par étudier le cas où K est convexe mais nous nous en éloignerons progressivement.

3.4.1 Algorithme de gradient à pas fixe avec projection

Nous nous limitons dans cette sous-section au cas où K est un convexe fermé non vide de V. Sous cette hypothèse, le Théorème 2.3.8 assure l'existence et l'unicité de la solution u de (3.55), caractérisée d'après le Théorème 2.5.1 par la condition d'optimalité

$$\langle J'(u), v-u\rangle \geq 0 \quad \forall\, v \in K\ . \tag{3.56}$$

Pour tout réel $\mu > 0$ (qui jouera le rôle d'un pas de descente), (3.56) s'écrit

$$\langle u - \big(u - \mu J'(u)\big), v-u\rangle \geq 0 \quad \forall\, v \in K\ . \tag{3.57}$$

Notons P_K l'opérateur de projection orthogonale sur l'ensemble convexe K, défini à la Remarque 5.1.4. D'après le Théorème 5.1.3 de projection sur un convexe, (3.57) n'est rien d'autre que la caractérisation de u comme la projection orthogonale de $u - \mu J'(u)$ sur K, c'est-à-dire,

$$u = P_K\big(u - \mu J'(u)\big)\ . \tag{3.58}$$

Plus précisément, (3.58) est en fait équivalent à (3.56) et caractérise donc la solution u de (3.55). L'algorithme de **gradient à pas fixe avec projection** (ou plus simplement de gradient projeté) est alors défini par l'itération

$$u^{n+1} = P_K\big(u^n - \mu J'(u^n)\big)\ , \tag{3.59}$$

où $\mu > 0$ est un pas de descente fixe. Autrement dit, (3.59) est une simple adaptation de l'algorithme de gradient à pas fixe auquel on a ajouté une étape de projection pour ne pas quitter l'ensemble admissible K.

Théorème 3.4.1 *On suppose que J est α-convexe différentiable et que J' est Lipschitzien sur V (de constante L, voir (3.9)). Alors, si $0 < \mu < 2\alpha/L^2$, l'algorithme de gradient à pas fixe avec projection converge : quel que soit $u^0 \in K$, la suite (u^n) définie par (3.59) converge vers la solution u de (3.55).*

Démonstration. La démonstration reprend celle du Théorème 3.2.6 où l'on a montré que l'application $v \mapsto v - \mu J'(v)$ est strictement contractante lorsque $0 < \mu < 2\alpha/L^2$, c'est-à-dire qu'il existe $\gamma \in]0,1[$ tel que

$$\forall v, w \in V, \quad \big\|\big(v - \mu J'(v)\big) - \big(w - \mu J'(w)\big)\big\| \leq \gamma \|v - w\|\ .$$

Puisque la projection P_K est faiblement contractante d'après (8.2), l'application $v \mapsto P_K\big(v - \mu J'(v)\big)$ est strictement contractante, ce qui prouve la convergence de la suite (u^n) définie par (3.59) vers la solution u de (3.55). □

Exercice 3.4.1 Soit $V = \mathbb{R}^N$ et $K = \{x \in \mathbb{R}^N$ tel que $\sum_{i=1}^N x_i = 1\}$. Expliciter l'opérateur de projection orthogonale P_K et interpréter dans ce cas la formule (3.58) en terme de multiplicateur de Lagrange.

Le Théorème 3.4.1 montre que l'algorithme de gradient à pas fixe avec projection est applicable à une large classe de problèmes d'optimisation convexe avec contraintes, dès lors que l'on sait calculer l'opérateur de projection orthogonale P_K. Mais malheureusement, il est très difficile, non seulement de connaitre explicitement l'opérateur de projection P_K, mais même de construire un algorithme simple et efficace pour l'évaluer, dans le cas d'un convexe fermé K quelconque dans V. Malgré sa simplicité apparente l'algorithme du gradient projeté est donc souvent un leurre du point de vue pratique par défaut d'une connaissance explicite de cet opérateur de projection P_K.

Donnons néanmoins deux exceptions importantes pour lesquelles on sait calculer explicitement P_K. Par souci de simplicité dans la présentation, on se limite au cas de la dimension finie $V = \mathbb{R}^N$, mais ces deux exemples se généralisent sans difficulté à des espaces de Hilbert de dimension infinie. On considère tout d'abord le cas d'un sous-espace affine

$$K = \{x \in \mathbb{R}^N \ Bx = c\},$$

avec B matrice $M \times N$ de rang maximal, $\text{rang}(B) = M \leq N$, et $c \in \mathbb{R}^M$. On trouve alors facilement que

$$P_K(x) = x + B^*(BB^*)^{-1}(c - Bx).$$

En effet, la condition d'optimalité du Théorème 2.5.1 (inéquation d'Euler) est en fait une égalité qui dit que $(x - P_K(x))$ est orthogonal à $\text{Ker}B$ donc appartient à $\text{Im}B^*$, c'est-à-dire $x - P_K(x) = B^*y$. Enfin, la condition $BP_K(x) = c$ permet de calculer $y = (BB^*)^{-1}(Bx - c)$.

Un second cas particulier important concerne les sous-ensembles K de la forme

$$K = \prod_{i=1}^{N} [a_i, b_i] \tag{3.60}$$

(avec éventuellement $a_i = -\infty$ ou $b_i = +\infty$ pour certains indices i). En effet, il est alors facile de voir que, si $x = (x_1, x_2, \ldots, x_N) \in \mathbb{R}^N$, $y = P_K(x)$ a pour composantes

$$y_i = \min\left(\max\left(a_i, x_i\right), b_i\right) \quad \text{pour} \quad 1 \leq i \leq N\,, \tag{3.61}$$

autrement dit, il suffit juste de "tronquer" les composantes de x.

Remarque 3.4.2 Ce qui rend très attractif, mais très difficile à mettre en oeuvre, l'algorithme du gradient projeté est qu'il s'agit d'un **algorithme faisable**, c'est-à-dire qu'à chaque itération la solution approchée u^n appartient bien à l'ensemble K qui définit les contraintes. A l'opposé de cette approche il existe une classe d'**algorithmes infaisables** où la suite de solutions approchées u^n n'appartient pas à K mais, en cas de convergence, leur limite u appartient bien à K. C'est évidemment beaucoup plus facile à mettre en oeuvre mais, en général, avant la convergence on n'a pas une solution, même de qualité médiocre, qui satisfait les contraintes. La section suivante donne des exemples de tels algorithmes infaisables. •

3.4.2 Algorithme d'Uzawa

L'idée de l'algorithme d'Uzawa vient de la théorie de la dualité, entrevue dans la Section 2.6, qui permet de "transformer" un problème avec contraintes d'inégalités générales en un problème "dual" avec comme seule contrainte d'appartenir au convexe $(\mathbb{R}_+)^M$, qui est du type (3.60). Grâce à la formule explicite (3.61) de projection sur ce convexe particulier, ce problème dual peut être résolu par la méthode du gradient à pas fixe avec projection. La solution du problème primal pourrait ensuite être obtenue, par exemple, en résolvant un problème de minimisation **sans contrainte** pour le Lagrangien. En fait, l'algorithme d'Uzawa est aussi une méthode de recherche de point-selle pour le Lagrangien et il calcule à la fois une solution du problème dual et une du problème primal.

Considérons donc le problème de minimisation convexe

$$\inf_{F(v)\leq 0} J(v) \ , \tag{3.62}$$

où J est une fonctionnelle convexe définie sur V et F une fonction convexe de V dans $\mathbb{R}^M$. Sous les hypothèses du Théorème de Kuhn et Tucker 2.6.4, la résolution de (3.62) revient à trouver un point-selle (u,p) du Lagrangien

$$\mathcal{L}(v,q) = J(v) + q \cdot F(v) \ , \tag{3.63}$$

sur $V \times (\mathbb{R}_+)^M$. A partir de la Définition 2.6.1 du point-selle

$$\forall\, q \in (\mathbb{R}_+)^M \quad \mathcal{L}(u,q) \leq \mathcal{L}(u,p) \leq \mathcal{L}(v,p) \quad \forall\, v \in V \ , \tag{3.64}$$

on déduit que $(p-q)\cdot F(u) \geq 0$ pour tout $q \in (\mathbb{R}_+)^M$, d'où on tire, pour tout réel $\mu > 0$,

$$(p-q)\cdot \big(p - \big(p + \mu F(u)\big)\big) \leq 0 \quad \forall\, q \in (\mathbb{R}_+)^M \ ,$$

ce qui, d'après (8.1), montre que

$$p = P_{\mathbb{R}^M_+}(p + \mu F(u)) \quad \forall\, \mu > 0 \ , \tag{3.65}$$

$P_{\mathbb{R}^M_+}$ désignant la projection de $\mathbb{R}^M$ sur $(\mathbb{R}_+)^M$.

Au vu de cette propriété et de la seconde inégalité dans (3.64), nous pouvons introduire **l'algorithme d'Uzawa** : à partir d'un élément quelconque $p^0 \in (\mathbb{R}_+)^M$, on construit les suites (u^n) et (p^n) déterminées par les itérations

$$\begin{aligned} &\mathcal{L}(u^n,p^n) = \inf_{v\in V} \mathcal{L}(v,p^n) \ , \\ &p^{n+1} = P_{\mathbb{R}^M_+}(p^n + \mu F(u^n)) \ , \end{aligned} \tag{3.66}$$

μ étant un paramètre positif fixé. On peut interpréter l'algorithme d'Uzawa en disant qu'alternativement il minimise le Lagrangien par rapport à v avec q fixé et il maximise (par un seul pas de l'algorithme du gradient projeté) ce même Lagrangien par rapport à q avec v fixé. Une autre manière de voir l'algorithme d'Uzawa est la suivante : il prédit une valeur du multiplicateur de Lagrange q et effectue

une minimisation sans contrainte du Lagrangien par rapport à v, puis il corrige la prédiction de q en l'augmentant si la contrainte est violée et en le diminuant sinon. Nous verrons une troisième interprétation de l'algorithme d'Uzawa dans le cadre de la théorie de la dualité ci-dessous.

Théorème 3.4.3 *On suppose que J est α-convexe différentiable, que F est convexe et Lipschitzienne de V dans $\mathbb{R}^M$, c'est-à-dire qu'il existe une constante L telle que*

$$\|F(v) - F(w)\| \leq L\|v - w\| \quad \forall\, v, w \in V \ , \tag{3.67}$$

Si $0 < \mu < 2\alpha/L^2$, l'algorithme d'Uzawa converge : quel que soit l'élément initial p^0, la suite (u^n) définie par (3.66) converge vers la solution u du problème (3.62).

Démonstration. Tout d'abord, l'existence et l'unicité d'une solution u de (3.62) découle du Théorème 2.3.8 puisque l'on minimise une fonction fortement convexe sur un convexe fermé non vide. Le Théorème de Kuhn et Tucker 2.6.4 affirme alors qu'il existe un multiplicateur de Lagrange $p \in (\mathbb{R}_+)^M$ tel que (u, p) est un point-selle du Lagrangien (3.63) sur $V \times (\mathbb{R}_+)^M$. De même, p^n étant fixé, le problème de minimisation dans (3.66) a bien une solution unique u^n. D'après l'Exercice 2.5.6, les inéquations d'Euler satisfaites par u et u^n s'écrivent

$$\langle J'(u), v - u\rangle + p \cdot \big(F(v) - F(u)\big) \geq 0 \quad \forall\, v \in V \ , \tag{3.68}$$

$$\langle J'(u^n), v - u^n\rangle + p^n \cdot \big(F(v) - F(u^n)\big) \geq 0 \quad \forall\, v \in V \ . \tag{3.69}$$

Prenant successivement $v = u^n$ dans (3.68) et $v = u$ dans (3.69) et additionnant, on obtient

$$\langle J'(u) - J'(u^n), u^n - u\rangle + (p - p^n) \cdot \big(F(u^n) - F(u)\big) \geq 0 \ ,$$

d'où en utilisant l'α-convexité de J et en posant $r^n = p^n - p$

$$r^n \cdot \big(F(u^n) - F(u)\big) \leq -\alpha\|u^n - u\|^2 \ . \tag{3.70}$$

D'autre part, la projection $P_{\mathbb{R}^M_+}$ étant faiblement contractante d'après (8.2), en soustrayant (3.65) à (3.66) on obtient

$$\|r^{n+1}\| \leq \|r^n + \mu(F(u^n) - F(u))\| \ ,$$

soit

$$\|r^{n+1}\|^2 \leq \|r^n\|^2 + 2\mu r^n \cdot \big(F(u^n) - F(u)\big) + \mu^2\|F(u^n) - F(u)\|^2 \ .$$

Utilisant (3.67) et (3.70), il vient

$$\|r^{n+1}\|^2 \leq \|r^n\|^2 + \big(L^2\mu^2 - 2\mu\alpha\big)\|u^n - u\|^2 \ .$$

Si $0 < \mu < 2\alpha/L^2$, on peut trouver $\beta > 0$ tel que $L^2\mu^2 - 2\mu\alpha < -\beta$, d'où

$$\beta\|u^n - u\|^2 \leq \|r^n\|^2 - \|r^{n+1}\|^2 \ . \tag{3.71}$$

Ceci montre alors que la suite $\|r^n\|^2$ est décroissante : le membre de droite de (3.71) tend donc vers 0, ce qui entraîne que u^n tend vers u. □

Ainsi, l'algorithme d'Uzawa permet d'approcher la solution de (3.62) en remplaçant ce problème avec contraintes par une suite de problèmes de minimisation sans contraintes (3.66). A chaque itération, la détermination de p^n est élémentaire, puisque d'après (3.61) l'opérateur de projection $P_{\mathbb{R}_+^M}$ est une simple troncature à zéro des composantes négatives. Il faut aussi noter que le Théorème 3.4.3 ne dit rien de la convergence de la suite (p^n). En fait, cette convergence n'est pas assurée sous les hypothèses du théorème, qui n'assurent d'ailleurs pas l'unicité de l'élément $p \in (\mathbb{R}_+)^M$ tel que (u, p) soit point-selle (voir la Remarque 2.6.13 et l'Exercice 3.4.2 ci-dessous).

Il reste à faire le lien entre l'algorithme d'Uzawa et la théorie de la dualité, comme nous l'avons déjà annoncé. Rappelons d'abord que le problème dual de (3.62) s'écrit

$$\sup_{q \geq 0} \mathcal{G}(q) \ , \tag{3.72}$$

où, par définition

$$\mathcal{G}(q) = \inf_{v \in V} \mathcal{L}(v, q) \ , \tag{3.73}$$

et que le multiplicateur de Lagrange p est une solution du problème dual (3.72). En fait, sous des hypothèses assez générales, on peut montrer que $\mathcal{G}$ est différentiable et que le gradient $\mathcal{G}'(q)$ est précisément égal à $F(u_q)$, où u_q est l'unique solution du problème de minimisation (3.73). En effet, on a

$$\mathcal{G}(q) = J(u_q) + q \cdot F(u_q),$$

et en dérivant formellement par rapport à q

$$\mathcal{G}'(q) = F(u_q) + \langle J'(u_q) + q \cdot F'(u_q), u_q' \rangle = F(u_q),$$

à cause de la condition d'optimalité pour u_q. On voit alors que **l'algorithme d'Uzawa n'est autre que la méthode du gradient à pas fixe avec projection appliquée au problème dual** puisque la deuxième équation de (3.66) peut s'écrire $p^{n+1} = P_{\mathbb{R}_+^M}\big(p^n + \mu \mathcal{G}'(p^n)\big)$ (le changement de signe par rapport à (3.59) vient du fait que le problème dual (3.72) est un problème de maximisation et non de minimisation). Le lecteur vérifiera très facilement cette assertion dans le cas particulier étudié à l'exercice suivant.

Exercice 3.4.2 Appliquer l'algorithme d'Uzawa au problème de la Remarque 2.6.13 (fonctionnelle quadratique et contraintes affines en dimension finie). Si la matrice B est de rang M, ce qui assure l'unicité de p d'après la Remarque 2.6.13, montrer que la suite p^n converge vers p.

Une variante, plus simple, de l'algorithme d'Uzawa est **l'algorithme d'Arrow-Hurwicz** qui s'interprète lui aussi comme un algorithme de point selle. Simplement, au lieu de minimiser exactement en v à chaque itération de (3.66), l'algorithme d'Arrow-Hurwicz effectue un seul pas d'une méthode de gradient. Concrètement, à

partir d'éléments quelconques $p^0 \in (\mathbb{R}_+)^M$ et $u^0 \in V$, on construit les suites (u^n) et (p^n) déterminées par les itérations

$$\begin{aligned} u^{n+1} &= u^n - \mu^n \left(J'(u^n) + p^n \cdot F'(u^n)\right) , \\ p^{n+1} &= P_{\mathbb{R}_+^M}(p^n + \mu^n F(u^n)) , \end{aligned} \tag{3.74}$$

où $\mu^n > 0$ est un pas de descente positif. Autrement dit, (3.74) recherche un point selle en alternant un pas de minimisation en v et un pas de maximisation en q.

Théorème 3.4.4 *On suppose que J est α-convexe différentiable, que F est convexe différentiable de dérivée bornée par $C_F > 0$. On suppose aussi que, pour tout $p \in (\mathbb{R}_+)^M$, la fonction $v \to J'(v) + p \cdot F'(v)$ est L-Lipschitzienne sur V. Alors il existe un point-selle (u, p) du Lagrangien (3.63) sur $V \times (\mathbb{R}_+)^M$ et u est la solution unique du problème (3.62). Si le pas de descente vérifie, pour $C^2 = \max(2L^2 + C_F^2, 2C_F^2)$,*

$$\mu^n = \frac{\alpha\gamma}{C^2} \frac{\|u^n - u\|^2}{\|u^n - u\|^2 + \|p^n - p\|^2} \qquad \textit{avec} \;\; 0 < \gamma < 1,$$

alors, l'algorithme d'Arrow-Hurwicz converge : quel que soit les éléments initiaux p^0, u^0, la suite (u^n) définie par (3.74) converge vers la solution u du problème (3.62).

Démonstration. Les hypothèses impliquent qu'il existe un point-selle (u, p) du Lagrangien (3.63) sur $V \times (\mathbb{R}_+)^M$. Rappelons que d'après (3.65) on a $p = P_{\mathbb{R}_+^M}(p + \mu F(u))$. Comme la projection P_K est faiblement contractante d'après (8.2), on déduit de la deuxième équation de (3.74) que

$$\begin{aligned} \|p^{n+1} - p\|^2 &\le \|p^n + \mu F(u^n) - (p + \mu F(u))\|^2 \\ &\le \|p^n - p\|^2 + 2\mu(p^n - p) \cdot (F(u^n) - F(u)) + \mu^2 \|F(u^n) - F(u)\|^2 \\ &\le \|p^n - p\|^2 + 2\mu(p^n - p) \cdot F(u^n) + \mu^2 C_F^2 \|u^n - u\|^2 \end{aligned} \tag{3.75}$$

car $p^n \cdot F(u) \le p \cdot F(u) = 0$ et la dérivée de F est supposée bornée par C_F. La première équation de (3.74) conduit à

$$\|u^{n+1} - u\|^2 \le \|u^n - u\|^2 - 2\mu \langle \frac{\partial \mathcal{L}}{\partial v}(u^n, p^n), (u^n - u) \rangle + \mu^2 \|\frac{\partial \mathcal{L}}{\partial v}(u^n, p^n)\|^2. \tag{3.76}$$

Or, comme $J'(u) + p \cdot F'(u) = 0$, on a

$$\frac{\partial \mathcal{L}}{\partial v}(u^n, p^n) = J'(u^n) + p \cdot F'(u^n) - (J'(u) + p \cdot F'(u)) + (p^n - p) \cdot F'(u^n),$$

donc

$$\begin{aligned} \|\frac{\partial \mathcal{L}}{\partial v}(u^n, p^n)\|^2 &\le 2 \left(\|J'(u^n) + p \cdot F'(u^n) - (J'(u) + p \cdot F'(u))\|^2 + \|(p^n - p) \cdot F'(u^n)\|^2 \right) \\ &\le 2 \left(L^2 \|u^n - u\|^2 + C_F^2 \|p^n - p\|^2 \right) , \end{aligned}$$

car $J'(v)+p\cdot F'(v)$ est L-Lipschitzienne et F' est borné. Par conséquent, en sommant (3.75) et (3.76) et en appliquant le Lemme 3.4.5 pour $v = u^n$ et $q = p^n$, on trouve

$$\begin{aligned}\|u^{n+1}-u\|^2+\|p^{n+1}-p\|^2 &\leq \|u^n-u\|^2+\|p^n-p\|^2-\mu\alpha\|u-u^n\|^2\\ &\quad+\mu^2C^2\left(\|u^n-u\|^2+\|p^n-p\|^2\right)\\ &\leq \|u^n-u\|^2+\|p^n-p\|^2-\frac{\gamma(1-\gamma)\alpha^2\|u^n-u\|^4}{C^2(\|u^n-u\|^2+\|p^n-p\|^2)}\end{aligned}$$

avec $C^2 = \max(2L^2 + C_F^2, 2C_F^2)$. On en déduit que la suite $\|u^n - u\|^2 + \|p^n - p\|^2$ est décroissante positive donc convergente, et par ailleurs que le rapport $\|u^n - u\|^4 / (\|u^n - u\|^2 + \|p^n - p\|^2)$ tends vers zéro. Si la suite $\|u^n - u\|^2 + \|p^n - p\|^2$ tends vers zéro, alors la suite (u^n, p^n) converge vers le point selle (u, p). Sinon, alors c'est la suite $\|u^n - u\|^4$ qui tends vers zéro, c'est-à-dire que la suite u^n converge vers u (mais on ne peut rien dire pour la suite p^n). □

Lemme 3.4.5 *Soit le Lagrangien $\mathcal{L}(v, q) = J(v) + q \cdot F(v)$, défini sur $V \times (\mathbb{R}_+)^M$, avec J et F convexes différentiables et J α-convexe. On suppose que (u, p) est un point selle du Lagrangien. Alors, pour tout $(v, q) \in V \times (\mathbb{R}_+)^M$,*

$$\frac{\alpha}{2}\|u-v\|^2 \leq \langle\frac{\partial\mathcal{L}}{\partial v}(v,q),(v-u)\rangle - \frac{\partial\mathcal{L}}{\partial q}(v,q)\cdot(q-p)\,.$$

Démonstration. Pour $q \geq 0$ le Lagrangien est α-convexe

$$\mathcal{L}(u,q) \geq \mathcal{L}(v,q) + \langle\frac{\partial\mathcal{L}}{\partial v}(v,q),(u-v)\rangle + \frac{\alpha}{2}\|u-v\|^2\,.$$

Comme (u, p) est un point selle, $\mathcal{L}(u, q) \leq \mathcal{L}(u, p)$ et on en déduit

$$\frac{\alpha}{2}\|u-v\|^2 \leq \langle\frac{\partial\mathcal{L}}{\partial v}(v,q),(v-u)\rangle + \mathcal{L}(u,p) - \mathcal{L}(v,q)\,. \tag{3.77}$$

Or $\mathcal{L}(v, q) = \mathcal{L}(v, p) + (q - p) \cdot F(v)$ et $\frac{\partial\mathcal{L}}{\partial q}(v, q) = F(v)$. La deuxième inégalité du point selle, $\mathcal{L}(u, p) \leq \mathcal{L}(v, p)$ permet enfin de conclure à partir de (3.77). □

3.4.3 Pénalisation des contraintes

Une autre méthode pour approcher un problème de minimisation avec contraintes par une suite de problèmes de minimisation sans contraintes est la procédure de **pénalisation** des contraintes. Il ne s'agit pas d'un algorithme précis mais plutôt d'une stratégie de résolution qui doit, pour sa partie numérique, utiliser l'un des algorithmes de la Sous-section 3.2 pour l'optimisation sans contrainte. Cette résolution numérique peut d'ailleurs soulever des difficultés, car le problème "pénalisé" est souvent "mal conditionné". Néanmoins l'avantage de cette méthode de pénalisation est sa facilité de mise en oeuvre, même si elle est parfois décevante pour la qualité de ses résultats numériques.

Nous nous plaçons pour simplifier dans le cas où $V = \mathbb{R}^N$, et nous considérons de nouveau le problème de minimisation convexe

$$\inf_{F(v)\leq 0} J(v) \ , \tag{3.78}$$

où J est une fonction convexe continue de $\mathbb{R}^N$ dans $\mathbb{R}$ et F une fonction convexe continue de $\mathbb{R}^N$ dans $\mathbb{R}^M$. Pour $\varepsilon > 0$, on introduit le problème sans contraintes

$$\inf_{v\in\mathbb{R}^N} \left(J(v) + \frac{1}{\varepsilon} \sum_{i=1}^{M} \left[\max\left(F_i(v), 0\right)\right]^2 \right) \ , \tag{3.79}$$

dans lequel ont dit que les contraintes $F_i(v) \leq 0$ sont "pénalisées". On peut alors énoncer le résultat suivant, qui montre que, pour ε petit, le problème (3.79) "approche bien" le problème (3.78).

Proposition 3.4.6 *On suppose que J est continue, strictement convexe, et infinie à l'infini, que les fonctions F_i sont convexes et continues pour $1 \leq i \leq M$, et que l'ensemble*

$$K = \left\{ v \in \mathbb{R}^N \quad , \quad F_i(v) \leq 0 \quad \forall\, i \in \{1, \ldots, M\} \right\}$$

est non vide. En notant u l'unique solution de (3.78) et, pour $\varepsilon > 0$, u_ε l'unique solution de (3.79), on a alors

$$\lim_{\varepsilon\to 0} u_\varepsilon = u \ .$$

Démonstration. L'ensemble K étant convexe fermé, l'existence et l'unicité de u découlent du Théorème 2.2.1 et de la stricte convexité de J. De plus, la fonction $G(v) = \sum_{i=1}^{M} \left[\max\left(F_i(v), 0\right)\right]^2$ est continue et convexe puisque la fonction de $\mathbb{R}$ dans $\mathbb{R}$ qui à x associé $\max(x, 0)^2$ est convexe et croissante. On en déduit que la fonctionnelle $J_\varepsilon(v) = J(v) + \varepsilon^{-1} G(v)$ est strictement convexe, continue, et infinie à l'infini puisque $G(v) \geq 0$, ce qui implique l'existence et l'unicité de u_ε. Comme $G(u) = 0$, on peut écrire

$$J_\varepsilon(u_\varepsilon) = J(u_\varepsilon) + \frac{G(u_\varepsilon)}{\varepsilon} \leq J_\varepsilon(u) = J(u) \ . \tag{3.80}$$

Ceci montre que

$$J(u_\varepsilon) \leq J_\varepsilon(u_\varepsilon) \leq J(u) \ , \tag{3.81}$$

et donc que u_ε est borné d'après la condition "infinie à l'infini". On peut donc extraire de la famille (u_ε) une suite (u_{ε_k}) qui converge vers une limite u_* lorsque ε_k tend vers 0. On a alors $0 \leq G(u_{\varepsilon_k}) \leq \varepsilon_k (J(u) - J(u_{\varepsilon_k}))$ d'après (3.80). Passant à la limite, on obtient $G(u_*) = 0$, qui montre que $u_* \in K$. Comme (3.81) implique que $J(u_*) \leq J(u)$, on a alors $u_* = u$, ce qui conclut la démonstration, toutes les suites extraites (u_{ε_k}) convergeant vers la même limite u. □

Exercice 3.4.3 En plus des hypothèses de la Proposition 3.4.6, on suppose que les fonctions J et $F_1, \ldots, F_M$ sont continûment différentiables. On note de nouveau $I(u)$ l'ensemble des contraintes actives en u, et on suppose que les contraintes sont qualifiées

en u au sens de la Définition 2.5.15. Enfin, on suppose que les vecteurs $\left(F_i'(u)\right)_{i\in I(u)}$ sont linéairement indépendants, ce qui assure l'unicité des multiplicateurs de Lagrange $\lambda_1,\dots,\lambda_M$ tels que $J'(u)+\sum_{i=1}^M \lambda_i F_i'(u)=0$, avec $\lambda_i=0$ si $i\notin I(u)$. Montrer alors que, pour tout indice $i\in\{1,\dots,M\}$

$$\lim_{\varepsilon\to 0}\left[\frac{2}{\varepsilon}\max\left(F_i(u_\varepsilon),0\right)\right]=\lambda_i\ .$$

En pratique, même si la Proposition 3.4.6 garantit que la solution u_ε est une bonne approximation de la solution exacte u pour ε suffisamment petit, on ne choisit pas des paramètres ε trop petits à cause d'un problème de conditionnement numérique. En effet, lorsqu'on évalue la fonction J_ε ou ses dérivées on additionne des termes d'ordre 1 par rapport à ε (qui viennent de J) et des termes d'ordre $1/\varepsilon$ (qui viennent de F), sans parler de la division de $\max(F_i(x),0)$ (qui peut être petit à convergence) par ε. Les erreurs d'arrondi peuvent être donc très marquées dans de tels calculs. Pour cette raison on a souvent recours à une **méthode de continuation** qui consiste à diminuer progressivement la valeur de ϵ. Ce problème de conditionnement numérique est en quelque sorte la face cachée de la facilité de mise en oeuvre pratique.

Remarque 3.4.7 L'approche proposée ci-dessus est une méthode de **pénalisation extérieure**, au sens où la solution pénalisée u_ε ne vérifie en général pas les contraintes. C'est uniquement sa limite qui les vérifie. A l'opposé de cette approche, il existe une technique de **pénalisation intérieure**, pour des contraintes d'inégalité, qui garantit à chaque itération que la solution pénalisée u_ε vérifie bien les contraintes, pour peu que l'initialisation les vérifie. Expliquons en le principe pour l'exemple du problème (3.78). Pour $\varepsilon>0$, nous introduisons le problème sans contraintes

$$\inf_{v\in\mathbb{R}^N, F(v)<0}\left(J(v)-\varepsilon\sum_{i=1}^M\frac{1}{F_i(v)}\right) \tag{3.82}$$

où le deuxième terme est appelée une fonction "barrière" dont le choix n'est pas unique. De manière générale, une fonction barrière est une fonction définie sur $\mathbb{R}_-$ qui tend vers $+\infty$ quand son argument tend vers zéro. Le but de cette barrière est d'empêcher une suite de solutions approchées de (3.82) de s'approcher de la limite $F_i(v)=0$. Ainsi les contraintes $F(v)<0$ ne sont jamais actives et peuvent être ignorées en pratique. Ainsi, si on utilise un algorithme de gradient pour le problème sans contrainte (3.82) avec une initialisation qui vérifie strictement les contraintes et un pas de descente suffisamment petit, alors la suite des itérées va rester dans le domaine strictement faisable $F(v)<0$. Pour que l'effet de la barrière soit de plus en plus négligeable, on fait progressivement tendre le paramètre $\varepsilon>0$ vers zéro (notez la différence fondamentale avec le coefficient $1/\varepsilon$ en pénalisation extérieure). Nous étudierons un exemple de pénalisation intérieure à la Sous-section 4.2.3. •

Exercice 3.4.4 Soit des fonctions J et $F_1,\dots,F_M$ convexes. Vérifiez que la fonction minimisée dans (3.82) est bien convexe sur le domaine défini par $F(v)<0$.

3.4.4 Algorithme du Lagrangien augmenté

Nous concluons cette sous-section en présentant l'algorithme du Lagrangien augmenté, qui est une méthode combinant les avantages de l'algorithme d'Uzawa et de la méthode de pénalisation. En particulier, il est nettement plus robuste que la méthode de pénalisation dont il évite le caractère "mal conditionné".

L'idée de la méthode est d'introduire un Lagrangien **augmenté** d'un terme de pénalisation des contraintes. Pour fixer les idées, plaçons nous en dimension finie, $V = \mathbb{R}^N$, et considérons le problème de minimisation avec contraintes d'égalité

$$\inf_{F(v)=0} J(v) \ , \tag{3.83}$$

où J est une fonction de $\mathbb{R}^N$ dans $\mathbb{R}$ et F une fonction de $\mathbb{R}^N$ dans $\mathbb{R}^M$, toutes les deux supposées de classe C^1. On remarque que le problème (3.83) est équivalent à

$$\inf_{F(v)=0} \left\{ J(v) + \frac{\mu}{2}\|F(v)\|^2 \right\},$$

pour tout paramètre $\mu \in \mathbb{R}$, puisque $F(v) = 0$. Cela suggère d'introduire, pour $v \in \mathbb{R}^N$, $\lambda \in \mathbb{R}^M$ et $\mu > 0$, le **Lagrangien augmenté**

$$\mathcal{L}_{aug}(v, \lambda, \mu) = J(v) + \lambda \cdot F(v) + \frac{\mu}{2}\|F(v)\|^2, \tag{3.84}$$

où $\|F(v)\|$ est la norme Euclidienne dans $\mathbb{R}^M$ du vecteur $F(v)$. Autrement dit, on additionne une fonction quadratique de la contrainte au Lagrangien habituel (3.63). Dans (3.84), λ est bien sûr un multiplicateur de Lagrange pour la contrainte, tandis que μ ressemble à un coefficient de pénalisation. Dans la sous-section précédente ce coefficient était noté $1/\varepsilon$. Ce changement de notation n'est pas anodin car, ici, il ne sera pas nécessaire en pratique de faire tendre μ vers l'infini pour que la contrainte soit satisfaite. En réalité, si l'interprétation du Lagrangien augmenté comme une méthode de pénalisation est commode, elle n'est pas tout à fait juste et il s'agit plutôt d'une méthode de régularisation du type Moreau-Yosida (voir l'Exercice 3.3.5) pour la fonction duale [10].

Le principe de l'algorithme est de trouver un point selle de (3.84). A λ et μ fixés, on minimise en v le Lagrangien augmenté par une des méthodes proposées pour la minimisation sans contrainte. A v et μ fixés, on maximise en λ par un algorithme itératif dont on va donner maintenant le principe. A l'itération n on note λ^n la valeur du multiplicateur de Lagrange et v^n le point de minimum de $v \to \mathcal{L}_{aug}(v, \lambda^n, \mu)$. Ecrivons la condition d'optimalité pour v^n

$$J'(v^n) + \lambda^n \cdot F'(v^n) + \mu F(v^n) \cdot F'(v^n) = 0. \tag{3.85}$$

Si le même vecteur v^n était solution de (3.83), on aurait la condition d'optimalité du Théorème 2.5.6

$$J'(v^n) + \lambda^* \cdot F'(v^n) = 0, \tag{3.86}$$

où λ^* est un multiplicateur de Lagrange optimal. En comparant (3.85) et (3.86) on obtient (en supposant que les contraintes sont qualifiées en v^n)

$$\lambda^* = \lambda^n + \mu F(v^n). \tag{3.87}$$

De (3.87) on tire deux informations. D'une part, si la suite λ^n tends vers λ^*, il n'est pas nécessaire de faire tendre le coefficient de pénalisation μ vers l'infini pour avoir la satisfaction progressive de la contrainte $F(v) = 0$. D'autre part, (3.87) suggère une formule de mise à jour du multiplicateur de Lagrange

$$\lambda^{n+1} = \lambda^n + \mu F(v^n).$$

Cette formule est très semblable à celle de l'algorithme d'Uzawa, voir (3.74), sauf qu'ici le pas μ n'est pas petit. Au total **l'algorithme du Lagrangien augmenté** est le suivant : on choisit $\mu > 0$, on initialise $\lambda_0 \in \mathbb{R}^M$ et on construit deux suites v^n et λ^n par

$$\mathcal{L}_{aug}(v^n, \lambda^n, \mu) = \min_{v \in \mathbb{R}^N} \mathcal{L}_{aug}(v, \lambda^n, \mu),$$
$$\lambda^{n+1} = \lambda^n + \mu F(v^n).$$

Par ailleurs, de temps en temps, et un nombre limité de fois (voir le chapitre 17 de [24]), on augmente la valeur du coefficient de pénalisation μ. Mais, comme le rappelle le résultat ci-dessous, il n'est pas nécessaire de faire tendre μ vers l'infini pour converger.

Lemme 3.4.8 *On suppose que les fonctions J et F sont de classe C^2. Soit v^* un point de minimum local de* (3.83) *où les contraintes sont qualifiées au sens que la matrice $F'(v^*)$ est de rang M. Soit λ^* un multiplicateur de Lagrange pour lequel la condition d'optimalité de* (3.83) *est vérifiée. On suppose que la condition suffisante d'optimalité d'ordre 2 est satisfaite en v^*, à savoir*

$$\left(J''(v^*) + \lambda^* \cdot F''(v^*)\right)(w, w) > 0 \quad \forall\, w \in K(v^*) = \mathrm{Ker} F'(v^*), w \neq 0. \tag{3.88}$$

Alors, il existe $\mu_0 > 0$ tel que, pour tout $\mu \geq \mu_0$, v^* est un point de minimum local de $v \to \mathcal{L}_{aug}(v, \lambda^*, \mu)$ (sans contrainte).

Remarque 3.4.9 L'intérêt du Lemme 3.4.8 est de montrer que, si l'on connait la valeur du multiplicateur de Lagrange optimal, alors la minimisation, sans contrainte et avec une pénalisation **finie** de la contrainte, du Lagrangien augmenté conduit à une solution du problème d'origine (3.83). Cela justifie, en quelque sorte, le fait qu'il n'est pas nécessaire de faire tendre vers l'infini le coefficient de pénalisation μ pour que l'algorithme du Lagrangien augmenté converge, du moins si on a une bonne estimation du multiplicateur de Lagrange. •

Démonstration. Si v^* un point de minimum local de (3.83) et que les contraintes sont qualifiées, alors la condition d'optimalité du premier ordre du Théorème 2.5.6 donne

$$J'(v^*) + \lambda^* \cdot F'(v^*) = 0 \quad \text{et} \quad F(v^*) = 0,$$

ce qui implique que $\mathcal{L}'_{aug}(v^*, \lambda^*, \mu) = 0$ (ici et dans toute la suite de cette preuve le signe $'$ indique une dérivée en v). Calculons la dérivée seconde en v du Lagrangien augmenté

$$\mathcal{L}''_{aug}(v^*, \lambda^*, \mu)(w, w) = \mathcal{L}''(v^*, \lambda^*)(w, w) + \mu \|F'(v^*)w\|^2, \tag{3.89}$$

avec

$$\mathcal{L}''(v^*, \lambda^*)(w, w) = J''(v^*)(w, w) + \lambda^* \cdot F''(v^*)(w, w)$$

et où il n'y a pas de dérivée seconde dans le terme de pénalisation car $F(v^*) = 0$. L'hypothèse (3.88) montre que le premier terme à droite dans (3.89) est une forme quadratique définie positive sur $\mathrm{Ker}F'(v^*)$, tandis que le second terme est une forme quadratique définie positive sur le sous-espace de dimension M engendré par les colonnes de $F'(v^*)$ (qui est aussi l'orthogonal de $\mathrm{Ker}F'(v^*)$). Notons que sur ce sous-espace de dimension fini $\mathcal{L}''_{aug}(v*, \lambda^*, \mu)(w, w)$ peut prendre des valeurs négatives mais qu'on peut les minorer par $-\mu_0\|w\|^2$ où $-\mu_0$ est la plus petite valeur propre de la matrice $M \times M$ qui représente cette forme quadratique sur ce sous-espace. Il suffit alors de choisir $\mu > \mu_0$ pour que la somme des deux termes soit strictement positif pour $w \neq 0$. Au total, la dérivée seconde du Lagrangien augmenté est définie positive en v^* qui est un point critique. C'est donc un point de minimum local. □

3.5 Méthode de Newton

La méthode de Netwon est un algorithme d'ordre deux pour la minimisation d'une fonction coût car elle utilise l'information des deux premières dérivées de cette fonction. On imagine qu'elle peut être plus efficace qu'un algorithme du premier ordre, puisqu'elle utilise, en plus du gradient, la Hessienne de la fonction, mais qu'elle risque d'être plus compliqué à mettre en oeuvre puisqu'il faut justement calculer ces dérivées secondes.

3.5.1 Cas de la dimension finie

Pour simplifier la présentation, on se place en dimension finie $V = \mathbb{R}^N$. On veut résoudre le problème de minimisation d'une fonction régulière $J(v)$ de $\mathbb{R}^N$ dans $\mathbb{R}$, sans contraintes. On sait que les éventuels points de minimum de $J(v)$ se trouvent parmi les zéros de la dérivée $J'(v)$. Le principe de la méthode de Newton est de chercher les zéros de la dérivée $J'(v)$. Remarquons tout de suite un inconvénient de ce principe : ces zéros peuvent aussi correspondre à des points de maximum ou des points selle et la méthode de Newton ne permet pas de faire le tri entre minima, maxima ou simples points stationnaires. Evidemment, si la fonction J est convexe, on sait qu'elle n'a que des points de minimum.

Rentrons dans les détails et notons $F = J'$. On suppose que F une fonction de classe C^2 de $\mathbb{R}^N$ dans $\mathbb{R}^N$. Soit u un zéro régulier de F c'est-à-dire que

$$F(u) = 0 \quad \text{et} \quad F'(u) \text{ matrice inversible.}$$

Une formule de Taylor au voisinage de v nous donne

$$F(u) = F(v) + F'(v)(u - v) + \mathcal{O}\left(\|u - v\|^2\right),$$

c'est-à-dire

$$u = v - \left(F'(v)\right)^{-1} F(v) + \mathcal{O}\left(\|v - u\|^2\right).$$

La méthode de Newton consiste à résoudre de façon itérative cette équation en négligeant le reste. Pour un choix initial $u^0 \in \mathbb{R}^N$, on calcule

$$u^{n+1} = u^n - \left(F'(u^n)\right)^{-1} F(u^n) \quad \text{pour} \quad n \geq 0. \tag{3.90}$$

Rappelons que l'on ne calcule pas l'inverse de la matrice $F'(u^n)$ dans (3.90) mais que l'on résout un système linéaire. Du point de vue de l'optimisation, la méthode de Newton s'interprète de la manière suivante. Soit J une fonction de classe C^3 de $\mathbb{R}^N$ dans $\mathbb{R}$, et soit u un minimum local de J. En notant $F = J'$, la méthode précédente permet de résoudre la condition nécessaire d'optimalité $J'(u) = 0$. Plus précisément, on peut aussi voir la méthode de Newton comme une méthode de minimisation. A cause du développement de Taylor

$$J(w) = J(v) + J'(v) \cdot (w - v) + \frac{1}{2} J''(v)(w - v) \cdot (w - v) + \mathcal{O}\left(\|w - v\|^3\right), \tag{3.91}$$

on peut approcher $J(w)$ au voisinage de v par une fonction quadratique. La méthode de Newton consiste alors à minimiser cette approximation quadratique et à itérer. Le minimum de la partie quadratique du terme de droite de (3.91) est donné par $w = v - \left(J''(v)\right)^{-1} J'(v)$ si la matrice $J''(v)$ est définie positive. On retrouve alors la formule itérative (3.90).

L'avantage principal de la méthode de Newton est sa convergence bien plus rapide que les méthodes précédentes.

Proposition 3.5.1 *Soit F une fonction de classe C^2 de $\mathbb{R}^N$ dans $\mathbb{R}^N$, et u un zéro régulier de F (i.e. $F(u) = 0$ et $F'(u)$ inversible). Il existe un réel $\epsilon > 0$ et une constante $0 < C < 1/\epsilon$ tels que, si u^0 est assez proche de u au sens où $\|u - u^0\| \leq \epsilon$, la méthode de Newton définie par (3.90) converge, c'est-à-dire que la suite (u^n) converge vers u, au sens où*

$$\|u^{n+1} - u\| \leq C\|u^n - u\|^2 \quad \text{et} \quad \|u^n - u\| \leq C^{-1}(C\epsilon)^{2^n}. \tag{3.92}$$

Remarque 3.5.2 La convergence de la méthode de Newton, dite **quadratique**, est extrêmement rapide (bien plus que la méthode du gradient qui converge simplement géométriquement d'après la Remarque 3.2.8). Le nombre de chiffres significatifs dans l'approximation u^n de la solution u double à chaque itération. Bien sûr, cette convergence rapide a un prix car, à chaque itération de la méthode de Newton (3.90), il faut résoudre un système linéaire, ce qui est coûteux en temps de calcul. De plus, la convergence rapide donnée par (3.92) n'a lieu que si F est de classe C^2, et si u^0 est assez proche de u, hypothèses bien plus restrictives que celles que nous avons utilisées jusqu'à présent. Effectivement, même dans des cas très simples dans $\mathbb{R}$, la méthode de Newton peut diverger pour certaines données initiales u^0 ; il faut noter aussi que la convergence quadratique (3.92) ne se produit qu'au voisinage d'un zéro régulier, comme le montre l'application de la méthode de Newton à la fonction $F(x) = \|x\|^2$ dans $\mathbb{R}^N$, pour laquelle la convergence n'est que géométrique. Par ailleurs, si on applique la méthode de Newton pour la minimisation d'une fonction J comme expliqué ci-dessus, il se peut que la méthode converge vers un maximum

ou un col de J, et non pas vers un minimum, car elle ne fait que rechercher les zéros de J'. La méthode de Newton n'est donc pas supérieure en tout point aux algorithmes précédents, mais la propriété de convergence locale quadratique (3.92) la rend cependant particulièrement intéressante. •

Démonstration. Par continuité de F' et de F'' il existe $\delta > 0$ tel que F' est inversible en tout point v de la boule de centre u et de rayon δ et, de plus, il existe $C_1, C_2 > 0$ tels que

$$\| \left(F'(v)\right)^{-1} \| \leq C_1, \quad \|F''(v)\| \leq C_2, \quad \text{pour tout } \|v-u\| \leq \delta.$$

Supposons que toutes les itérées jusqu'à u^n sont restées proches de u, au sens où $\|u-u^n\| \leq \delta$, donc $F'(u^n)$ est inversible. Comme $F(u)=0$, on déduit de (3.90)

$$u^{n+1} - u = u^n - u - \left(F'(u^n)\right)^{-1} \left(F(u^n) - F(u)\right)$$

qui, par développement de Taylor autour de u^n, devient

$$u^{n+1} - u = \frac{1}{2} \left(F'(u^n)\right)^{-1} \left(\int_0^1 F''(u^n + s(u-u^n))\, dx \right) (u^n - u) \cdot (u^n - u).$$

On peut majorer pour obtenir

$$\|u^{n+1} - u\| \leq \frac{1}{2} C_1 C_2 \|u^n - u\|^2.$$

qui n'est rien d'autre que la première partie de (3.92) avec $C = C_1C_2/2$. On choisit $0 < \epsilon < \min(\delta, C^{-1})$ et $\|u-u^0\| \leq \epsilon$. On vérifie par récurrence que $\|u-u^n\| \leq \epsilon \leq \delta$ (donc toutes les itérées restent proches de u). En prenant le logarithme de l'inégalité précédente, on a

$$\log \|u^{n+1} - u\| \leq \log C + 2 \log \|u^n - u\|,$$

d'où l'on déduit

$$\log \|u^n - u\| \leq 2^n \log \|u^0 - u\| + (2^n - 1) \log C$$

c'est-à-dire

$$\|u^n - u\| \leq C^{-1} \Big(C \|u^0 - u\| \Big)^{2^n} \leq C^{-1} (C\epsilon)^{2^n}$$

qui converge vers zéro puisque $C\epsilon < 1$. □

Remarque 3.5.3 Un inconvénient majeur de la méthode de Newton est la nécessité de connaître le Hessien $J''(v)$ (ou la matrice dérivée $F'(v)$). Lorsque le problème est de grande taille ou bien si J n'est pas facilement deux fois dérivable, on peut modifier la méthode de Newton pour éviter de calculer cette matrice $J''(v) = F'(v)$. Les méthodes, dites de quasi-Newton, proposent de calculer de façon itérative aussi une approximation S^n de $\left(F'(u^n)\right)^{-1}$. On remplace alors la formule (3.90) par

$$u^{n+1} = u^n - S^n F(u^n) \quad \text{pour} \quad n \geq 0.$$

En général on calcule S^n par une formule de récurrence du type

$$S^{n+1} = S^n + C^n$$

où C^n est une matrice de rang 1 qui dépend de $u^n, u^{n+1}, F(u^n), F(u^{n+1})$, choisie de manière à ce que $S^n - (F'(u^n))^{-1}$ converge vers 0. Pour plus de détails sur ces méthodes de quasi-Newton nous renvoyons à [6] et [13]. •

On peut adapter la méthode de Newton à la minimisation d'une fonction J avec des contraintes d'égalité. Soit J une fonction de classe C^3 de $\mathbb{R}^N$ dans $\mathbb{R}$, $G = (G_1, ..., G_M)$ une fonction de classe C^3 de $\mathbb{R}^N$ dans $\mathbb{R}^M$ (avec $M \leq N$), et soit u un minimum local de

$$\min_{v \in \mathbb{R}^N,\ G(v)=0} J(v)\ . \tag{3.93}$$

Si les vecteurs $(G_1'(u), ..., G_M'(u))$ sont linéairement indépendants, la condition nécessaire d'optimalité du Théorème 2.5.6 est

$$J'(u) + \sum_{i=1}^{M} \lambda_i G_i'(u) = 0\ , \quad G_i(u) = 0 \quad 1 \leq i \leq M\ . \tag{3.94}$$

où les $\lambda_1, \ldots, \lambda_M \in \mathbb{R}$ sont les multiplicateurs de Lagrange. On peut alors résoudre le système (3.94) de $(N + M)$ équations à $(N + M)$ inconnues $(u, \lambda) \in \mathbb{R}^{N+M}$ par une méthode de Newton. On pose donc

$$F(u, \lambda) = \begin{pmatrix} J'(u) + \lambda \cdot G'(u) \\ G(u) \end{pmatrix},$$

dont la matrice dérivée est

$$F'(u, \lambda) = \begin{pmatrix} J''(u) + \lambda \cdot G''(u) & (G'(u))^* \\ G'(u) & 0 \end{pmatrix}.$$

On peut alors appliquer l'algorithme de Newton (3.90) à cette fonction $F(u, \lambda)$ si la matrice $F'(u, \lambda)$ est inversible. Nous allons voir que cette condition est "naturelle" au sens où elle correspond à une version un peu plus forte de la condition d'optimalité d'ordre 2 de la Proposition 2.5.13. La matrice $F'(u, \lambda)$ est inversible si elle est injective. Soit (w, μ) un élément de son noyau

$$\begin{cases} J''(u)w + \lambda \cdot G''(u)w + (G'(u))^*\mu = 0 \\ G_i'(u) \cdot w = 0 \text{ pour } 1 \leq i \leq M \end{cases}$$

On en déduit que $w \in \mathrm{Ker}G'(u) = \bigcap_{i=1}^{M} \mathrm{Ker}G_i'(u)$ et $(J''(u) + \lambda \cdot G''(u))w \in \mathrm{Im}(G'(u))^*$. Or $\mathrm{Im}(G'(u))^* = [\mathrm{Ker}G'(u)]^{\perp}$. Par conséquent, si on suppose que

$$(J''(u) + \lambda \cdot G''(u))\,(w, w) > 0 \quad \forall\, w \in \mathrm{Ker}G'(u), w \neq 0\ , \tag{3.95}$$

la matrice $F'(u, \lambda)$ est inversible. On remarque que (3.95) est l'inégalité stricte dans la condition d'optimalité d'ordre 2 de la Proposition 2.5.13. Il est donc naturel de

faire l'hypothèse (3.95) qui permet d'utiliser l'algorithme de Newton. On peut ainsi démontrer la convergence de cette méthode (voir [6]). Il est intéressant d'interpréter cet algorithme comme une méthode de minimisation. On introduit le Lagrangien $\mathcal{L}(v,\mu) = J(v) + \mu \cdot G(v)$, ses dérivées par rapport à v, $\mathcal{L}'$ et $\mathcal{L}''$, et on vérifie que l'équation

$$(u^{n+1}, \lambda^{n+1}) = (u^n, \lambda^n) - \left(F'(u^n, \lambda^n)\right)^{-1} F(u^n, \lambda^n)$$

est la condition d'optimalité pour que u^{n+1} soit un point de minimum du problème quadratique à contraintes affines

$$\min_{\substack{w\in\mathbb{R}^N \\ G(u^n)+G'(u^n)\cdot(w-u^n)=0}} Q^n(w)\ , \tag{3.96}$$

avec

$$Q^n(w) = \left(\mathcal{L}(u^n,\lambda^n) + \mathcal{L}'(u^n,\lambda^n)\cdot(w-u^n) + \frac{1}{2}\mathcal{L}''(u^n,\lambda^n)(w-u^n)\cdot(w-u^n)\right)\ ,$$

et λ^{n+1} est le multiplicateur de Lagrange associé au point de minimum de (3.96). On remarque que dans (3.96) on a effectué un développement de Taylor à l'ordre deux en w sur le Lagrangien $\mathcal{L}(w,\lambda^n)$ et on a linéarisé la contrainte $G(w)$ autour du point u^n.

Remarque 3.5.4 Dans (3.96) on a utilisé une approximation quadratique du Lagrangien et non pas de la fonction J. On pourrait essayer de se contenter d'une méthode itérative de résolution de l'approximation quadratique à contraintes affines suivante

$$\min_{\substack{w\in\mathbb{R}^N \\ G(v)+G'(v)\cdot(w-v)=0}} \left(J(v) + J'(v)\cdot(w-v) + \frac{1}{2}J''(v)(w-v)\cdot(w-v)\right)\ . \tag{3.97}$$

Malheureusement la méthode basée sur (3.97) peut ne pas converger ! En particulier, il n'est pas évident que le Hessien $J''(v)$ soit définie positif sur l'espace des contraintes (c'est le Hessien du Lagrangien qui est positif comme l'affirme la condition d'optimalité d'ordre 2 de la Proposition 2.5.13). •

3.5.2 Méthode de Gauss-Newton

Il s'agit d'une variante de l'algorithme Newton pour les problèmes de moindres carrés non-linéaires, qui s'écrivent

$$\min_{u\in\mathbb{R}^N} \|F(u)\|^2 \quad \text{avec } F(u) = (F_1(u), \cdots, F_M(u)),$$

où la fonction $F : \mathbb{R}^N \to \mathbb{R}^M$ est supposée de classe C^2. Si $F(u) = Au - b$, on retrouve les moindres carrés linéaires. L'idée de la méthode de Gauss-Newton est, à partir d'une initialisation $u^0 \in \mathbb{R}^N$, de construire une suite de solutions approchées u^n, pour $n \geq 0$. Comme pour l'algorithme de Newton, à chaque itération n, $F(u)$ est approchée par son application linéaire tangente en u^n, à savoir

$$F(u) \approx F(u^n) + \nabla F(u^n)(u-u^n) = A^n u - b^n,$$

où $A^n = \nabla F(u^n)$ est une matrice $M \times N$ et $b^n = \nabla F(u^n)u^n - F(u^n)$ est un vecteur de $\mathbb{R}^M$. On calcule alors la nouvelle itérée u^{n+1} comme un point de minimum de

$$\min_{u\in\mathbb{R}^N} \|A^n u - b^n\|^2,$$

c'est-à-dire que u^{n+1} est une solution de l'équation normale, $(\nabla F(u^n))^*\nabla F(u^n)u^n = (\nabla F(u^n))^* b^n$. Si l'on suppose que $\mathrm{Ker}\nabla F(u^n) = \{0\}$, alors la matrice $(\nabla F(u^n))^*\nabla F(u^n)$ est inversible et on vérifie aisément que u^{n+1} est donnée par

$$u^{n+1} = u^n - \Big((\nabla F(u^n))^*\nabla F(u^n)\Big)^{-1}(\nabla F(u^n))^* F(u^n)$$

S'il existe un zéro régulier u de F et si l'initialisation u^0 en est proche, on peut démontrer la convergence quadratique de l'algorithme de Gauss-Newton vers u.

Remarque 3.5.5 Si $N = M$ et $\mathrm{Ker}\nabla F(u^n) = \{0\}$, alors $\nabla F(u^n)$ est une matrice inversible et la formule ci-dessus se simplifie en

$$u^{n+1} = u^n - \Big(\nabla F(u^n)\Big)^{-1} F(u^n)$$

et on retrouve exactement l'algorithme de Newton pour trouver les solutions de $F(u) = 0$. •

3.5.3 Cas de la dimension infinie

Expliquons rapidement comment la méthode de Newton s'applique à la minimisation de fonctions définies sur un espace de Hilbert V de dimension infinie. Considérons une fonction convexe, de classe C^2, $J(v)$ de V dans $\mathbb{R}$. On repart du développement de Taylor à l'ordre 2 (3.91) et, pour une initialisation $u^0 \in V$, on calcule u^{n+1} comme un point de minimum de l'approximation quadratique

$$J^n(w) = J(u^n) + J'(u^n)(w - u^n) + \frac{1}{2}J''(u^n)\Big((w - u^n), (w - u^n)\Big), \tag{3.98}$$

où $V \times V \ni (w_1, w_2) \to J''(u^n)(w_1, w_2)$ est une forme bilinéaire symétrique. Si on suppose de plus qu'elle est coercive et continue, c'est-à-dire, s'il existe deux constantes $0 < \nu < M$ telles que

$$\nu\|w\|^2 \le J''(u^n)(w, w) \le M\|w\|^2,$$

alors la fonction $J^n(w)$, définie par (3.98), est fortement convexe, donc admet un unique point de minimum u^{n+1} caractérisé par la condition d'optimalité du premier ordre

$$(J^n)'(u^{n+1})(w) = 0 \quad \text{pour tout } w \in V,$$

que l'on peut réécrire : trouver $u^{n+1} \in V$, solution de

$$J''(u^n)\Big((u^{n+1} - u^n), w\Big) = -J'(u^n)(w) \quad \text{pour tout } w \in V. \tag{3.99}$$

L'équation (3.99) n'est rien d'autre qu'une formulation variationnelle sur V qu'on peut résoudre à l'aide du lemme de Lax-Milgram grâce à l'hypothèse de coercivité de $J''(u^n)$ (voir [1]). C'est typiquement de cette façon que sont résolues les équations aux dérivées partielles non-linéaires.

3.6 Méthodes d'approximations successives

Considérons un problème général d'optimisation sous contraintes d'égalité

$$\inf_{F(v)=0} J(v), \tag{3.100}$$

où $J(v)$ et $(F_1(v), ..., F_M(v)) = F(v)$ sont des fonctions régulières de $\mathbb{R}^N$ dans $\mathbb{R}$. Les remarques qui suivent s'appliquent de la même manière aux problèmes avec contraintes d'inégalité, moyennant des modifications évidentes.

Si l'application directe des algorithmes d'optimisation proposés dans ce chapitre est trop compliquée ou coûteuse pour (3.100), une stratégie courante consiste à remplacer ce dernier par une succession de problèmes approchés, obtenus, par exemple, par développement de Taylor des fonctions J et F autour de la solution approchée précédente. L'idée sous-jacente est qu'il est plus facile de résoudre le problème approché que le problème exact. Ces approximations n'ayant en général qu'un caractère local, il faut itérer cette stratégie en faisant un nouveau développement de Taylor au point de minimum obtenu sur le précédent problème approché.

Une méthode d'approximation successive peut donc se présenter comme suit. Etant donnée une initialisation $v^0 \in V = \mathbb{R}^N$, on construit une suite de solutions approchées $v^n \in V$, $n \geq 1$, où v^n est le point de minimum du problème

$$\inf_{F_{n-1}(v)=0} J_{n-1}(v), \tag{3.101}$$

où F_{n-1} et J_{n-1} sont des approximations de F et J, respectivement, au voisinage de la solution précédente v^{n-1}. Evidemment, il faut choisir F_{n-1} et J_{n-1}, non seulement pour que (3.101) admette une solution unique v^n, mais aussi pour que le calcul de v^n soit facile.

Expliquons le principe de cette méthode dans un cas très simple, en l'absence de contraintes d'égalité, c'est-à-dire $F \equiv 0$. On choisit $F_{n-1} \equiv 0$ et

$$J_{n-1}(v) = J(v^{n-1}) + J'(v^{n-1}) \cdot (v - v^{n-1}) + \frac{1}{2\mu}\|v - v^{n-1}\|^2, \tag{3.102}$$

avec un paramètre $\mu > 0$. Notons que les fonctions J et J_{n-1} sont tangentes en v^{n-1} et donc que J_{n-1} est bien une approximation de J en ce point.

Lemme 3.6.1 *Soit J une fonction différentiable de V dans $\mathbb{R}$ et $F \equiv 0$. Alors la méthode des approximations successives avec le choix* (3.102) *n'est rien d'autre que l'algorithme du gradient à pas fixe μ.*

Démonstration. Tout d'abord, la formule (3.102) est une approximation de Taylor au premier ordre de la fonction objectif J au point v^{n-1} à laquelle on a ajouté un terme quadratique de manière ce que J_{n-1} soit une fonction fortement convexe qui admet donc un unique point de minimum v^n. On écrit alors la condition d'optimalité nécessaire et suffisante, $J'_{n-1}(v^n) = 0$, qui donne

$$J'(v^{n-1}) + \frac{1}{\mu}(v^n - v^{n-1}) = 0,$$

ce qui est précisément la formule (3.8) de l'algorithme du gradient à pas fixe. □

Evidemment le Lemme 3.6.1 n'apporte rien de concret à l'algorithme du gradient à pas fixe mais il donne un point de vue nouveau qui permet des généralisations dont nous présentons deux exemples dans ce qui suit.

3.6.1 Programmation linéaire séquentielle

La première méthode, dite de **programmation linéaire successive** (ou séquentielle), consiste à remplacer les fonctions J et F par des approximations affines (cette méthode est connue aussi sous l'acronyme SLP pour l'anglais "sequential linear programming"). Etant donné une initialisation $v^0 \in \mathbb{R}^N$ (ne vérifiant pas nécessairement la contrainte $F(v) = 0$), on calcule une suite de solutions approchées v^n, $n \geq 1$, définies comme les solutions de

$$\inf_{F(v^{n-1})+F'(v^{n-1})\cdot(v-v^{n-1})=0} \Big\{ J(v^{n-1}) + J'(v^{n-1}) \cdot (v - v^{n-1}) \Big\}, \tag{3.103}$$

qui n'est rien d'autre qu'un programme linéaire, comme étudié dans la Section 4.2 et pour lequel on dispose d'algorithmes extrêmement efficaces. Une difficulté immédiate dans la résolution de (3.103) est que sa valeur minimum peut être $-\infty$ et qu'il n'y a pas de solution optimale. Notons que, sous une condition de qualification standard, $(F_1'(v^{n-1}), ..., F_M'(v^{n-1}))$ famille libre de $\mathbb{R}^N$, l'ensemble admissible de (3.103) n'est pas vide. C'est pourquoi, en pratique, cette méthode s'accompagne d'une contrainte supplémentaire, dite de **région de confiance**, qui prend la forme

$$\|v - v^{n-1}\| \leq \delta, \tag{3.104}$$

où $\delta > 0$ est un paramètre qui définit la taille du voisinage de v^{n-1} dans lequel (3.103) est une bonne approximation de (3.100). La norme dans (3.104) peut-être soit la norme $\|v\|_\infty = \max_{1\leq i\leq N} |v_i|$, soit la norme $\|v\|_1 = \sum_{i=1}^N |v_i|$, ce qui dans les deux cas préserve le fait que le problème approché est un programme linéaire. Ce dernier a alors nécessairement au moins une solution optimale puisque l'ensemble admissible est désormais borné.

3.6.2 Programmation quadratique séquentielle

Une deuxième méthode, dite de **programmation quadratique séquentielle**, consiste à remplacer la fonction J par une approximation quadratique et F par une approximation affine (cette méthode est connue aussi sous l'acronyme SQP pour l'anglais "sequential quadratic programming"). Etant donné une initialisation $v^0 \in \mathbb{R}^N$ (ne vérifiant pas nécessairement la contrainte $F(v) = 0$), on calcule une suite de solutions approchées v^n, $n \geq 1$, définies comme les solutions de

$$\inf_{F(v^{n-1})+F'(v^{n-1})\cdot(v-v^{n-1})=0} \Big\{ J(v^{n-1}) + J'(v^{n-1})\cdot(v-v^{n-1}) + \frac{1}{2} Q^{n-1}(v-v^{n-1})\cdot(v-v^{n-1}) \Big\}, \tag{3.105}$$

où Q^{n-1} est une matrice symétrique de taille N. Si Q^{n-1} est définie positive, alors on sait résoudre explicitement le problème (3.105) (voir l'Exercice 2.5.10). Le point crucial dans cette méthode SQP est que Q^{n-1} **n'est pas** la Hessienne de la fonction objectif $J''(v^{n-1})$ mais est la Hessienne du Lagrangien

$$Q^{n-1} = J''(v^{n-1}) + \lambda^{n-1} \cdot F''(v^{n-1}),$$

où λ^{n-1} est le multiplicateur de Lagrange dans la condition d'optimalité pour v^{n-1} (solution à l'itération précédente). En effet, ce qui importe n'est pas l'approximation de J par son développement de Taylor à l'ordre 2 dans tout $\mathbb{R}^N$ mais seulement sur la variété définit par la contrainte $F(v) = 0$. Concrêtement, on écrit

$$J(v) \approx J(v^{n-1}) + J'(v^{n-1}) \cdot (v - v^{n-1}) + \frac{1}{2} J''(v^{n-1})(v - v^{n-1}) \cdot (v - v^{n-1}) \quad (3.106)$$

et

$$0 = F(v) \approx F(v^{n-1}) + F'(v^{n-1}) \cdot (v - v^{n-1}) + \frac{1}{2} F''(v^{n-1})(v - v^{n-1}) \cdot (v - v^{n-1}), \quad (3.107)$$

puis on multiplie (3.107) par le multiplicateur de Lagrange λ^{n-1} et on somme le résultat à (3.106), ce qui donne exactement la fonction objectif de (3.105) (à une constante près, en utilisant la contrainte linéaire). La Proposition 2.5.13 (condition d'optimalité du 2ème ordre) montre que la matrice Q^{n-1} est positive si v^{n-1} est le point de minimum de (3.100), ce qui entraine que (3.105) admet au moins une solution optimale. Par contre, la matrice $J''(v^{n-1})$ n'a aucune raison d'être positive en général. Néanmoins, si v^{n-1} n'est pas un point de minimum, il peut être nécessaire de recourir à nouveau à une contrainte de région de confiance du type de (3.104). Pour plus de détails nous renvoyons à [24].

3.7 Modélisation, structures et algorithmes spécifiques

Nous terminons ce chapitre en évoquant quelques cas particuliers où se mêlent des questions de modélisation et d'algorithmique. Nous n'avons presque rien dit de la modélisation et pourtant il s'agit d'un aspect pratique très important en optimisation. Un problème d'optimisation n'est pas toujours présenté sous une forme mathématique claire et le praticien doit lui-même mettre en forme la question d'optimisation à laquelle il doit répondre. Comme toujours en modélisation, il s'agit de faire un compromis entre un modèle très précis et réaliste, mais intractable du point de vue algorithmique, et un modèle simplifié mais facilement optimisable en pratique. Dans cette étape de modélisation il est essentiel de faire émerger une structure du problème qui permette une mise en oeuvre efficace des algorithmes d'optimisation. Nous avons déjà vu des exemples de structures particulières qui autorisent des algorithmes efficaces (par exemple, un problème linéaire se traite avec l'algorithme du simplexe, voir le Chapitre 4) et nous allons voir ici trois cas particuliers où l'on peut

concevoir des méthodes qui tirent partie de la structure du problème pour résoudre efficacement leur optimisation. Bien sûr, nous ne sommes pas exhaustifs et nous ne voulons pas non plus donner l'impression de faire une liste de "recettes" : notre but est juste de montrer qu'il faut parfois un peu de réflexion avant de choisir tel ou tel algorithme plutôt qu'un autre. Dans le même ordre d'idée nous favorisons l'explication des idées à leur traitement mathématique rigoureux et complet.

3.7.1 Fonctions composées et rétro-propagation du gradient

On considère un problème d'optimisation sans contraintes où la fonction objectif J est la composée de m fonctions J_i

$$\inf_{v\in\mathbb{R}^N} J(v) = J_m \circ J_{m-1} \circ \cdots \circ J_1(v), \tag{3.108}$$

où chaque fonction $J_i(v_i)$, $1 \leq i \leq m$, est définie de $\mathbb{R}^{N_i}$ dans $\mathbb{R}^{N_{i+1}}$, avec $N_1 = N$ et $N_{m+1} = 1$ de sorte que J est une fonction de $\mathbb{R}^N$ dans $\mathbb{R}$. On suppose que chacune des fonctions J_i est différentiable. On se propose d'appliquer l'algorithme de gradient à pas fixe pour minimiser (3.108). Il faut donc calculer le gradient de la fonction composée J de façon efficace d'un point de vue numérique.

Cette structure particulière de fonction composée apparait dans au moins deux classes de problèmes importants en pratique. En premier lieu c'est une situation qu'on rencontre en apprentissage machine ou en intelligence artificielle lorsqu'on utilise des réseaux de neurones profonds. Chaque couche du réseau de neurones correspond à une fonction J_i. Dans la phase d'entrainement du réseau on résout le problème (3.108) où l'on optimise les coefficients des fonctions de transfer. Par souci de simplicité, on a supposé dans (3.108) que seuls les coefficients de la première couche sont optimisables. Bien entendu, on peut facilement généraliser à l'optimisation des coefficients de toutes les couches du réseau mais les notations deviennent plus lourdes et cachent la simplicité de l'idée que nous allons présenter. En second lieu, la fonction composée $J(v)$ est un modèle d'un programme informatique qui permet de calculer une quantité d'intérêt à l'aide de m sous-programmes qui s'enchainent comme la composition des m fonctions J_i. Le domaine de la différentiation automatique, à mi-chemin entre mathématiques et informatique, se donne pour but d'analyse le programme correspondant à $J(v)$ et de produire un autre programme qui fournit automatiquement le gradient $J'(v)$. Evidemment l'intérêt de cette approche est de pouvoir ensuite appliquer des algorithmes d'optimisation à la fonction J dont on n'a pas une expression analytique mais seulement un programme de calcul.

Rappelons le théorème de dérivation composée dans ce cas précis.

Lemme 3.7.1 *On note $v_i = J_i \circ \cdots \circ J_1(v)$ pour $1 \leq i \leq m$. La fonction $J(v) = J_m \circ J_{m-1} \circ \cdots \circ J_1(v)$ est différentiable au sens de Fréchet en tout $v \in \mathbb{R}^N$ et vérifie pour tout $h \in \mathbb{R}^N$*

$$J(v+h) = J(v) + J'_m(v_m) \cdot \big(J'_{m-1}(v_{m-1}) \cdots J'_1(v_1)h\big) + o(h), \tag{3.109}$$

où $J'_i(v_i)$ est une matrice $N_{i+1} \times N_i$, pour $1 \leq i \leq m-1$.

Remarque 3.7.2 Dans la formule (3.109) l'écriture $J'_{m-1}(v_{m-1}) \cdots J'_1(v_1)$ correspond à la multiplication des matrices successives. Une autre façon d'écrire le résultat du Lemme 3.7.1 est

$$J'(v) = (J_m \circ J_{m-1} \circ \cdots \circ J_1)'(v) = J'_m(v_m) J'_{m-1}(v_{m-1}) \cdots J'_1(v_1),$$

mais cela cache le fait, crucial pour la suite, que la dernière fonction J_m est à valeurs réelles, c'est-à-dire que son gradient est un vecteur. Notons que nous commettons un abus de notation car $J'_m(v_m)$ est un vecteur ligne dans l'équation ci-dessus alors que c'est un vecteur colonne dans le Lemme 3.7.1. •

Démonstration. Commençons par prouver le résultat dans le cas $m = 2$. Pour $v \in \mathbb{R}^N$ et $v_2 \in \mathbb{R}^{N_2}$, la différentiabilité de J_1 et J_2 s'écrit

$$\begin{cases} J_1(v+h) = J_1(v) + J'_1(v)h + o(h) & \text{pour tout } h \in \mathbb{R}^N, \\ J_2(v_2+h_2) = J_2(v_2) + J'_2(v_2) \cdot h_2 + o(h_2) & \text{pour tout } h_2 \in \mathbb{R}^{N_2}, \end{cases}$$

où $J'_1(v)$ est une matrice $N_2 \times N$ et $J'_2(v_2)$ est un vecteur de $\mathbb{R}^{N_2}$ puisque J_2 est une fonction de $\mathbb{R}^{N_2}$ dans $\mathbb{R}$. En posant $v_2 = J_1(v)$, $h_2 = J'_1(v)h + o(h)$ et en composant les deux dérivées on obtient

$$J_2 \circ J_1(v+h) = J_2 \circ J_1(v) + J'_2 \circ J_1(v) \cdot J'_1(v)h + o(h) \quad \text{pour tout } h \in \mathbb{R}^N.$$

Une simple récurrence permet d'obtenir le résultat pour tout m. □

Analysons la formule (3.109). Si l'on souhaite calculer la dérivée directionnelle de J dans une direction h donnée, alors cette formule est tout à fait convenable. On calcule $J'(v) \cdot h$ dans le même ordre que l'on calcule $J(v)$, c'est-à-dire en multipliant les contributions des gradients $J'_i(v_i)$ dans l'ordre croissant des indices i. Néanmoins, pour appliquer un algorithme de gradient, il faut connaitre le gradient complet, c'est-à-dire calculer toutes ses composantes qui sont les dérivées directionnelles suivant les vecteurs $(e_i)_{1 \le i \le N}$ de la base canonique de $\mathbb{R}^N$. Autrement dit, il faut appliquer N fois la formule (3.109), avec $h = e_i$, pour obtenir le gradient de J, ce qui est très inefficace.

L'algorithme de **rétro-propagation du gradient** se propose, non pas de calculer le gradient $J'(v)$, mais de fournir une direction de descente à moindre coût. Pour cela, on utilise le produit scalaire introduit dans (3.109) : on passe les matrices gradients de l'autre coté du produit scalaire en prenant leur adjoint et en inversant leur ordre de multiplication, ce qui conduit à

$$J'_m(v_m) \cdot \big(J'_{m-1}(v_{m-1}) \cdots J'_1(v_1)h\big) = \big(J'_1(v_1)^* J'_2(v_3)^* \cdots J'_{m-1}(v_{m-1})^* J'_m(v_m)\big) \cdot h,$$

où l'on rappelle que l'exposant $*$ désigne la matrice adjointe ou transposée. Autrement dit, on vient de trouver une direction de descente

$$h_{\text{desc}} = -J'_1(v_1)^* J'_2(v_3)^* \cdots J'_{m-1}(v_{m-1})^* J'_m(v_m), \tag{3.110}$$

qui garantit la décroissance de J pour peu qu'on la multiplie par un petit pas de descente $\mu > 0$. En effet, on vérifie que

$$J(v + \mu h_{\text{desc}}) = J(v) - \mu \|h_{\text{desc}}\|^2 + o(\mu).$$

L'algorithme de rétro-propagation du gradient est donc l'algorithme du gradient à pas fixe où l'on remplace le gradient par $-h_{\text{desc}}$, donné par (3.110). Son nom vient de ce qu'à l'inverse du calcul de $J(v)$ qui procède par composition des J_i dans l'ordre croissant des indices, ici on part du dernier gradient que l'on multiplie successivement par les gradients transposés dans l'ordre décroissant des indices. Le fait que cela soit les matrices adjointes des gradients qui jouent un rôle lui donne aussi le nom de méthode adjointe (au moins dans le domaine de la différentiation automatique). L'intérêt de cet algorithme est que le calcul de la direction de descente (3.110) est de la même complexité que le calcul d'une seule dérivée directionnelle de J, donc très économe en temps de calcul.

3.7.2 Optimisation sous contrainte de modèle et état adjoint

On considère maintenant une classe de problèmes d'optimisation où la variable d'optimisation est une combinaison de deux variables : $v \in \mathbb{R}^N$ la variable des paramètres de conception ou de contrôle d'un système et $y \in V$ la variable d'état décrivant ce système, avec un espace de Hilbert V. En fait, v et y sont reliés par une contrainte d'égalité, $F(v, y) = 0$, qui s'interprète en disant que l'état y du système est déterminé par les paramètres v. Cette contrainte d'égalité est typiquement un modèle qui permet de calculer y en fonction de v. Par exemple, y est la solution d'une équation différentielle où v est un paramètre (un coefficient, un terme source, etc.). Nous rencontrerons cette situation en théorie du contrôle optimal. En tout état de cause, il faut comprendre que la résolution du modèle pour trouver y en fonction de v est une opération non explicite et coûteuse.

Etant donné une fonction objectif $J(v, y)$, différentiable de $\mathbb{R}^N \times V$ dans $\mathbb{R}$, et une fonction contrainte $F(v, y)$, différentiable de $\mathbb{R}^N \times V$ dans V, on considère le problème d'optimisation

$$\inf_{v \in \mathbb{R}^N,\, y \in V \text{ tel que } F(v,y)=0} J(v, y). \tag{3.111}$$

Bien sûr, il est possible d'avoir d'autres contraintes mais nous nous concentrons ici sur la contrainte de modèle qui relie y à v. Nous faisons l'hypothèse suivante sur ce modèle.

Hypothèse (H). *Pour tout $v \in \mathbb{R}^N$, il existe une unique solution $y(v) \in V$ de la contrainte $F(v, y) = 0$, cette solution $v \to y(v)$ est différentiable de $\mathbb{R}^N$ dans V et l'opérateur linéarisé $w \to \frac{\partial F}{\partial y}(v, y(v))(w)$ est continu, inversible de V dans V et d'inverse continu de V dans V.*

Sous cette hypothèse, le problème (3.111) est équivalent au problème sans contrainte

$$\inf_{v \in \mathbb{R}^N} \widetilde{J}(v) = J(v, y(v)). \tag{3.112}$$

Il est conceptuellement facile de mettre en oeuvre un algorithme de gradient pour résoudre (3.112) mais nous allons voir que c'est une approche très inefficace dès que la dimension N est grande.

Lemme 3.7.3 *Sous l'hypothèse (H) la fonction $\widetilde{J}$ est différentiable sur $\mathbb{R}^N$ et, pour tout $h \in \mathbb{R}^N$, on a*

$$\widetilde{J}'(v) \cdot h = \frac{\partial J}{\partial v}(v, y(v)) \cdot h + \left\langle \frac{\partial J}{\partial y}(v, y(v)) \, , \, y'(v) \cdot h \right\rangle ,$$

où $\langle , \rangle$ désigne le produit scalaire de V et $y'(v) \cdot h$ est la dérivée de $v \to y(v)$ dans la direction h qui vérifie

$$\frac{\partial F}{\partial y}(v, y(v))(y'(v) \cdot h) = -\frac{\partial F}{\partial v}(v, y(v)) \cdot h. \tag{3.113}$$

Démonstration. C'est une simple application du théorème de dérivation composée où l'on doit juste noter que $y'(v) \cdot h$ et $\frac{\partial F}{\partial v}(v, y(v)) \cdot h$ appartiennent à V. Le système linéarisé (3.113) admet alors une solution unique $y'(v) \cdot h$ grâce à l'hypothèse (H). □

Remarque 3.7.4 Pour bien comprendre la portée du Lemme 3.7.3 et de l'hypothèse (H), on considère l'exemple suivant : $V = \mathbb{R}^M$ et $F(v, y) = Ay - b(v)$ où A est une matrice inversible $M \times M$ et $b(v)$ est une fonction régulière de $\mathbb{R}^N$ dans $\mathbb{R}^M$. Dans ce cas $\frac{\partial F}{\partial y}(v, y(v)) = A$ et l'hypothèse (H) est vérifiée car A est inversible et (3.113) est un simple système linéaire. Cet exemple est tout à fait représentatif car il correspond, par exemple, à la discrétisation d'une équation différentielle linéaire dont la solution y dépend d'une variable de contrôle v. La difficulté avec le Lemme 3.7.3 est qu'il ne donne pas une formule pour le gradient $\widetilde{J}'(v)$ mais seulement pour la dérivée directionnelle dans la direction h. Pour obtenir le gradient il faut calculer chacune de ses composantes en résolvant (3.113) pour h successivement égal à chacun des vecteurs e_i de la base canonique de $\mathbb{R}^N$, ce qui revient à résoudre N fois ce système linéaire, opération trop couteuse dans de nombreux cas pratiques où M et N sont grands. •

Pour dépasser la difficulté mentionnée ci-dessus et trouver une formule explicite, et économique, du gradient $\widetilde{J}'(v)$ nous allons introduire la notion **d'état adjoint** qui est en fait un multiplicateur de Lagrange pour la contrainte $F(v, y) = 0$. Pour tout $(v, y, p) \in \mathbb{R}^N \times V \times V$ on définit le Lagrangien

$$\mathcal{L}(v, y, p) = J(v, y) + \langle p, F(v, y) \rangle.$$

Le Lagrangien est affine en p et sa dérivée partielle par rapport à p n'est rien d'autre que la contrainte $F(v, y)$. Cela motive l'étude d'une autre dérivée partielle du Lagrangien, celle par rapport à y, qui vaut, pour tout $z \in V$,

$$\langle \frac{\partial \mathcal{L}}{\partial y}(v, y, p), z \rangle = \langle \frac{\partial J}{\partial y}(v, y), z \rangle + \langle p, \frac{\partial F}{\partial y}(v, y)(z) \rangle. \tag{3.114}$$

Lemme 3.7.5 *Le* **problème adjoint** *est défini par : trouver $p \in V$ solution de*

$$\left(\frac{\partial F}{\partial y}(v, y(v)) \right)^* p = -\frac{\partial J}{\partial y}(v, y(v)). \tag{3.115}$$

Sous l'hypothèse (H), ce problème admet une unique solution $p \equiv p(v) \in V$, appelé **état adjoint**. *Autrement dit, l'état adjoint est l'unique $p(v) \in V$ tel que*

$$\langle \frac{\partial \mathcal{L}}{\partial y}(v, y(v), p(v)), z\rangle = 0 \quad \text{pour tout } z \in V.$$

Démonstration. Comme $\frac{\partial F}{\partial y}(v, y(v))$ est un opérateur linéaire continu de V dans V, il admet un adjoint, noté $\left(\frac{\partial F}{\partial y}(v, y(v))\right)^*$. D'après l'hypothèse (H) il est de plus inversible d'inverse continu, et c'est vrai aussi pour son adjoint. Puisque J est différentiable, le second membre de (3.115) appartient à V et on en déduit donc l'existence et l'unicité de l'état adjoint $p(v)$ solution de (3.115). On remplace alors y par $y(v)$ et p par $p(v)$ dans (3.114) pour obtenir que la dérivée partielle du Lagrangien par rapport à y s'annule au point $(v, y(y), p(v))$. □

Proposition 3.7.6 *Sous l'hypothèse (H) la fonction $\widetilde{J}$ est différentiable sur $\mathbb{R}^N$ et on a*

$$\widetilde{J}'(v) = \frac{\partial J}{\partial v}(v, y(v)) + \langle p(v), \frac{\partial F}{\partial v}(v, y(v))\rangle, \tag{3.116}$$

où $p(v) \in V$ est **l'état adjoint**.

Remarque 3.7.7 La Proposition 3.7.6 est une amélioration considérable du Lemme 3.7.3 dès que le nombre de variables N est grand. En effet, la formule (3.116) donne le gradient complet de $\widetilde{J}$ au prix d'un seul calcul supplémentaire, celui de l'état adjoint $p(v)$. Au contraire, le Lemme 3.7.3 ne donnait qu'une dérivée directionnelle et il fallait résoudre N problèmes linéaires (3.113) (de même complexité que le problème adjoint) pour obtenir le gradient complet. Pour continuer sur l'exemple de la Remarque 3.7.5, où $V = \mathbb{R}^M$ et $F(v, y) = Ay - b(v)$ avec A une matrice inversible $M \times M$, le problème adjoint est un simple système linéaire à résoudre avec la matrice adjointe A^*. •

Démonstration. La contrainte étant satisfaite en $y(v)$, pour tout $p \in V$, on a

$$\widetilde{J}(v) = \mathcal{L}(v, y(v), p).$$

On dérive cette égalité par rapport à v pour obtenir

$$\widetilde{J}'(v) \cdot h = \frac{\partial \mathcal{L}}{\partial v}(v, y(v), p) \cdot h + \langle \frac{\partial \mathcal{L}}{\partial y}(v, y(v), p), (y'(v) \cdot h)\rangle.$$

On remplace alors p par l'état adjoint $p(v)$ dont la définition est précisément que $\langle \frac{\partial \mathcal{L}}{\partial y}(v, y(v), p(v)), z\rangle = 0$ pour tout $z \in V$. Par conséquent, on en déduit

$$\widetilde{J}'(v) \cdot h = \frac{\partial \mathcal{L}}{\partial v}(v, y(v), p(v)) \cdot h,$$

ce qui donne la formule (3.116). □

3.7.3 Décomposition-coordination

On considère un problème d'optimisation sous contraintes d'égalité

$$\inf_{v\in\mathbb{R}^N,\ F(v)=F_0} J(v), \tag{3.117}$$

où J est une fonction de $\mathbb{R}^N$ dans $\mathbb{R}$, F est une fonction de $\mathbb{R}^N$ dans $\mathbb{R}^M$ et $F_0 \in \mathbb{R}^M$ est un niveau de contrainte donné. On suppose que le problème est **décomposable**, c'est-à-dire qu'il existe une partition de la variable $v = (v_1, ..., v_n)$, avec $v_i \in \mathbb{R}^{n_i}$ et $\sum_{i=1}^n n_i = N$, et des fonctions objectifs et contraintes

$$J(v) = \sum_{i=1}^n J_i(v_i) \quad \text{et} \quad F(v) = \sum_{i=1}^n F_i(v_i),$$

de telle façon que le problème (3.117) est en fait équivalent à

$$\inf_{v_i\in\mathbb{R}^{n_i}} \sum_{i=1}^n J_i(v_i) \quad \text{sous la contrainte } \sum_{i=1}^N F_i(v_i) = F_0. \tag{3.118}$$

De nombreux problèmes pratiques peuvent se mettre sous cette forme et, pour fixer les idées, nous allons considérer le cas de l'optimisation du fonctionnement d'une entreprise composée de n divisions ou unités qui ont chacune leur propre variable de décision v_i et qui doivent minimiser leur coût de fonctionnement $J_i(v_i)$ avec leur contrainte de production $F_i(v_i)$, indépendants de celui des autres unités $j \neq i$, mais en partageant le niveau global de contraintes imposées, comme un budget, des ressources humaines ou des niveaux de production qui sont agrégées au niveau de l'entreprise entière.

Si on note p le multiplicateur de Lagrange associé à la contrainte $F(v) = F_0$, pour $p \in \mathbb{R}^M$ et $v \in \mathbb{R}^N$, on introduit le Lagrangien

$$\mathcal{L}(v, p) = J(v) + p \cdot (F(v) - F_0) = \sum_{i=1}^n (J_i(y_i) + p \cdot F_i(v)) - p \cdot F_0.$$

L'algorithme d'Uzawa (3.66) appliqué au problème (3.117) consiste, pour une initialisation $p_0 \in \mathbb{R}^M$, à construire les suites (u^k) et (p^k) déterminées par les itérations, pour $k \geq 0$,

$$\begin{aligned} \mathcal{L}(u^k, p^k) &= \inf_{v\in\mathbb{R}^N} \mathcal{L}(v, p^k)\,, \\ p^{k+1} &= p^k + \mu(F(u^k) - F_0)\,, \end{aligned} \tag{3.119}$$

où $\mu > 0$ est un pas positif fixé. L'hypothèse de décomposition des fonctions J et F rend en fait la minimisation du Lagrangien, à p fixé, particulièrement facile car il suffit de construire $u^k = (u_1^k, ..., u_n^k)$, où chaque composante u_i^k est la solution d'un problème de minimisation

$$J_i(u_i^k) + p^k \cdot F_i(u_i^k) = \inf_{v_i\in\mathbb{R}^{n_i}} \left(J_i(v_i) + p^k \cdot F_i(v_i)\right). \tag{3.120}$$

Chacun des problèmes (3.120) est de plus petite taille que (3.117) et indépendant des autres composantes $j \neq i$ du problème. Dans ce contexte, l'algorithme d'Uzawa est appelé algorithme de **décomposition par les prix**.

Une interprétation usuelle de cet algorithme est la suivante. Supposons, pour simplifier, que les fonctions J_i et F_i sont positives : elles correspondent au coût de production et au niveau de production, respectivement, de l'unité i. A chaque itération k, un organe central de l'entreprise fixe un prix $-p^k$ pour la valeur de la production (le vecteur p^k a toutes ses composantes négatives). Du coup, chaque unité veut maximiser son gain, c'est-à-dire minimiser son coût moins ses revenus correspondant au prix $-p^k$ de ce qu'elle produit. Autrement dit, chaque unité résout le problème (3.120) sans se soucier des autres unités. A la fin de l'iteration k, la direction de l'entreprise regarde si elle a atteint le niveau de production souhaité F_0. Si c'est le cas, la procédure a convergée. Sinon, on corrige les prix en les augmentant si le niveau de production est trop bas, $F(u^k) < F_0$, ou en les diminuant si la production est trop forte, $F(u^k) > F_0$ (on rappelle que le prix est $-p^k$).

Remarque 3.7.8 Nous ne disons rien de la convergence de cet algorithme de décomposition par les prix et nous renvoyons à des ouvrages plus spécialisés, comme [10]. Néanmoins, si les contraintes $F_i(v_i)$ sont affines et si les fonctions coûts $J_i(v_i)$ sont fortement convexes, alors le Théorème 3.4.3 affirme que l'algorithme d'Uzawa converge, c'est-à-dire que l'algorithme de décomposition par les prix converge dans ce cas particulier. On suppose ici que $F(v)$ est affine, ce qui permet de remplacer la contrainte d'égalité par deux contraintes (opposées) d'inégalité, qui sont toutes les deux convexes. •

Une autre approche de décomposition applicable au problème (3.118) est la **décomposition par les quantités**. Expliquons en le principe en reprenant l'exemple de l'entreprise composée de n unités. Il s'agit toujours d'un algorithme itératif et à chaque itération k la direction de l'entreprise confie à chaque unité i le soin de produire une quantité $F_i^k \in \mathbb{R}^M$, en ayant pris soin de choisir ses quantités qui vérifient la contrainte globale

$$\sum_{i=1}^{n} F_i^k = F_0.$$

Chaque unité doit donc résoudre un problème d'optimisation sous contrainte

$$\inf_{v_i \in \mathbb{R}^{n_i},\ F_i(v_i)=F_i^k} J_i(v_i), \tag{3.121}$$

dont on suppose qu'il admet une solution unique $u_i^k \in \mathbb{R}^{n_i}$. On suppose aussi qu'il existe un unique multiplicateur de Lagrange $p_i^k \in \mathbb{R}^M$ pour la contrainte dans (3.121). On note $G_i(F_i^k)$ la valeur du minimum dans (3.121), qui dépend du niveau de contraintes $F_i^k \in \mathbb{R}^M$. On sait par le Lemme 2.5.12 que, si la fonction G_i est différentiable, alors on a

$$p_i^k = -\nabla G_i(F_i^k).$$

Autrement dit, la sensibilité de la valeur du minimum par rapport au niveau de contraintes est égale à l'opposé du multiplicateur de Lagrange (qui s'interprète classiquement comme l'opposé d'un prix marginal). Ainsi, si deux unités $i \neq j$ ont

des multiplicateurs de Lagrange différents $p_i^k \neq p_j^k$, alors la direction de l'entreprise peut changer l'allocation de production à ces deux unités, en $F_i^k - \mu(p_j^k - p_i^k)$ et $F_j^k + \mu(p_j^k - p_i^k)$, pour $\mu > 0$ petit, ce qui conduit à une diminution de la somme des minima dans (3.121) pour i et j qui vaut au premier ordre $-\mu|p_j^k - p_i^k|^2 < 0$. On en conclut que tant que les multiplicateurs de Lagrange des unités sont différents, l'entreprise peut améliorer son minimum en changeant l'allocation des quantités à produire. Concrètement, pour passer à l'itération $(k+1)$, la direction de l'entreprise forme le prix moyen $p^{k+1} = \sum_{i=1}^n p_i^k/n$ et propose la nouvelle allocation

$$F_i^{k+1} = F_i^k + \mu(p_i^k - p^{k+1}).$$

On vérifie aisément que la contrainte globale $\sum_{i=1}^n F_i^{k+1} = F_0$ est toujours vérifiée et que, pour $\mu > 0$ petit, le gain dans le minimum de (3.118) est égal au premier ordre à

$$-\mu \sum_{i=1}^n |p_i^k - p^{k+1}|^2.$$

On améliore donc systématiquement le minimum avec cette méthode de décomposition par les quantités qui, par ailleurs, vérifie toujours la contrainte $F(u^k) = F_0$ au cours des itérations, contrairement à la méthode de décomposition par les prix (voir [10] pour plus de détails).